

Research Software Engineering

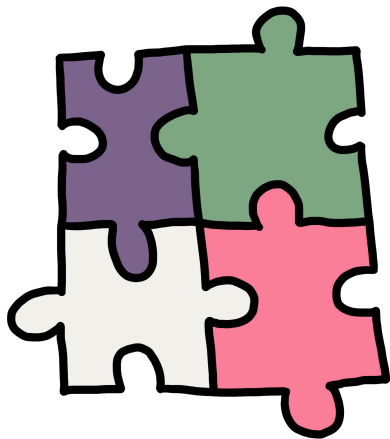
DAY 1



DIGITAL RESEARCH
ACADEMY

Dr. Fritjof Lammers

2-day workshop



Day 1

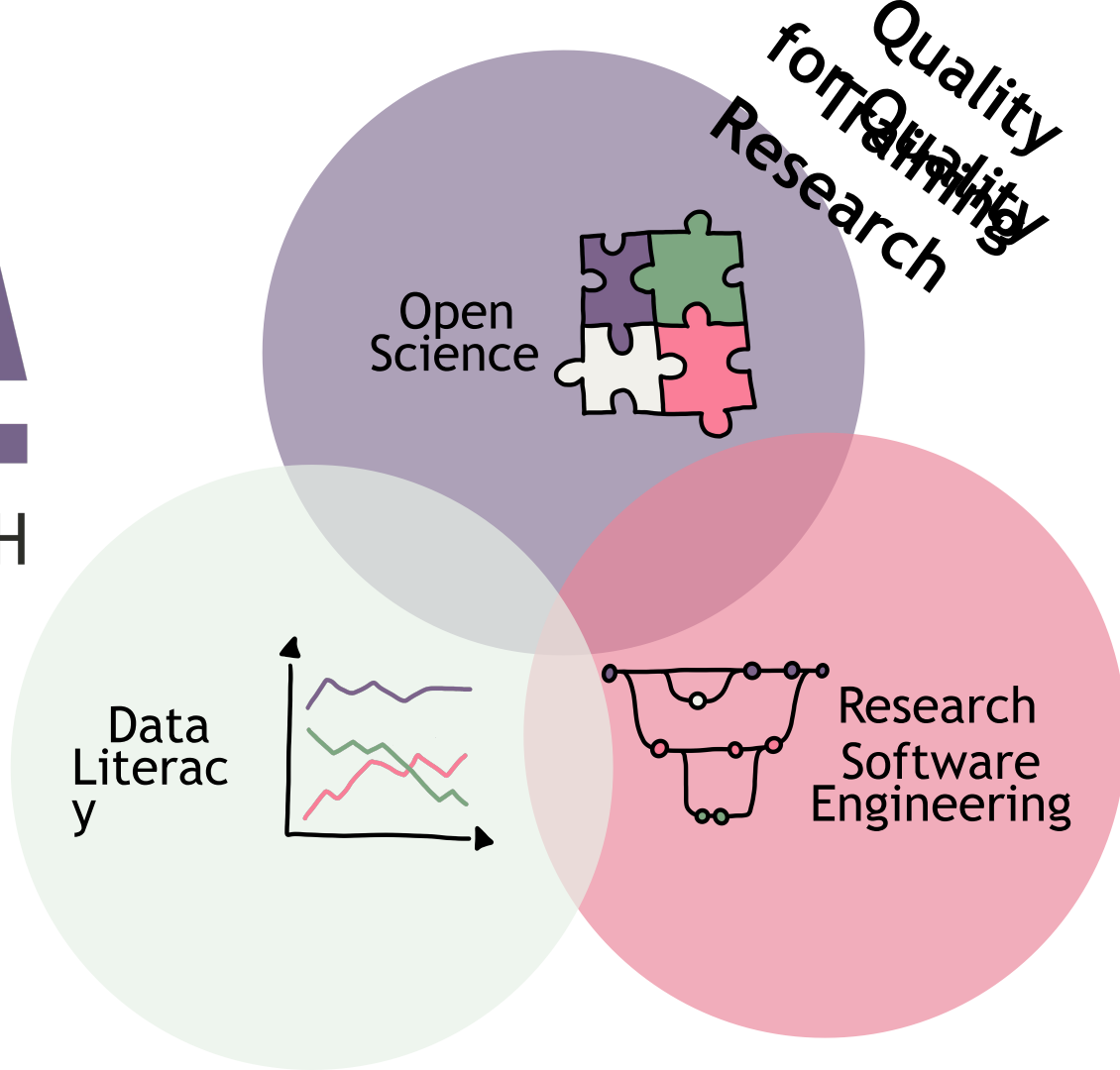
- What is Research Software Engineering
- Collaborative coding with Git & Github
- Reproducible coding

Day 2

- Introduction to testing
- Continuous Integration and Delivery (CI/CD)
- Software publication and licensing

DRA

DIGITAL RESEARCH
ACADEMY



Code of Conduct

Everyone is welcome, make everyone feel welcome, be nice.

<https://digital-research.academy/code-of-conduct/>



Please ask questions

<https://pad.okfn.de/p/rse-fzj-202511>

Your hopes and fears

Write down your thoughts...

The (main) outcome you'd like to see from this workshop.

Your main fear/worry about this workshop.

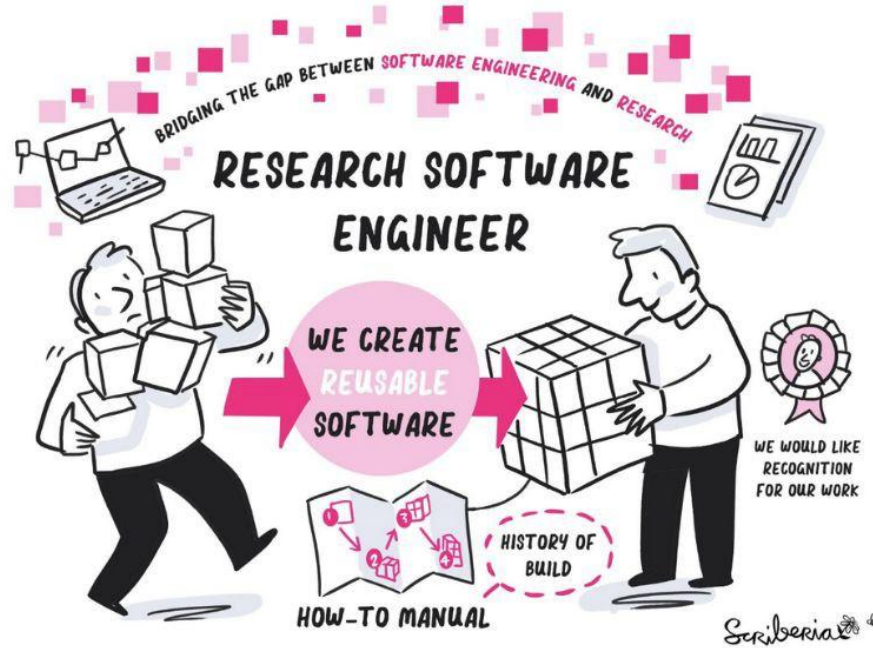
Who are you and why are you here?

Discussion 1

What is a Research
Software Engineer?



What is a Research Software Engineer?



What is a Research Software Engineer?

Definition is quite imprecise and notes from the original meeting where the term was coined (UK, 2012), include more a description of what it NOT is:

Roles: “research officer” – considered support don’t want to be servants

NOT permanent postdoc

Not a lab technician

Calling them software engineers is not good enough – domain specialised

“Researcher” has pressure to publish

<https://www.software.ac.uk/blog/not-so-brief-history-research-software-engineers-0>

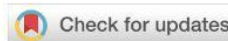
What is a Research Software Engineer?

... **the research software engineer**. These individuals combine a *professional attitude* to the exercise of software engineering with a *deep understanding of research topics*.

*Computational work must reflect the committed attitude of experimentalists towards caring about precise, professional, repeatable, meticulous work - **no-one with the same casual attitude to experimental instrumentation as many researchers have to code would be allowed anywhere near a lab**. This is striking considering how often research results now depend on software.*

Baxter et al. 2012

What is a Research Software Engineer?



OPINION ARTICLE

Foundational Competencies and Responsibilities of a Research Software Engineer

[version 1; peer review: awaiting peer review]

Florian Goth ¹, Renato Alves ², Matthias Braun³, Leyla Jael Castro ⁴,
Gerasimos Chourdakis ^{5,6}, Simon Christ ⁷, Jeremy Cohen ⁸,
Stephan Druskat ⁹, Fredo Erxleben¹⁰, Jean-Noël Grad ¹¹, Magnus Hagdorn ¹²,
Toby Hodges¹³, Guido Juckeland ¹⁰, Dominic Kempf ¹⁴,
Anna-Lena Lamprecht ¹⁵, Jan Linxweiler ¹⁶, Frank Löffler ¹⁷,
Michele Martone ¹⁸, Moritz Schwarzmeier ¹⁹, Heidi Seibold ²⁰,
Jan Philipp Thiele ^{21,22}, Harald von Waldow ²³, Samantha Wittke²⁴

What is a Research Software Engineer?

Training/teaching

Work in research environment

Produce software

Communicators/Translators

self-identify

Produce papers

Research Software Engineer

funding

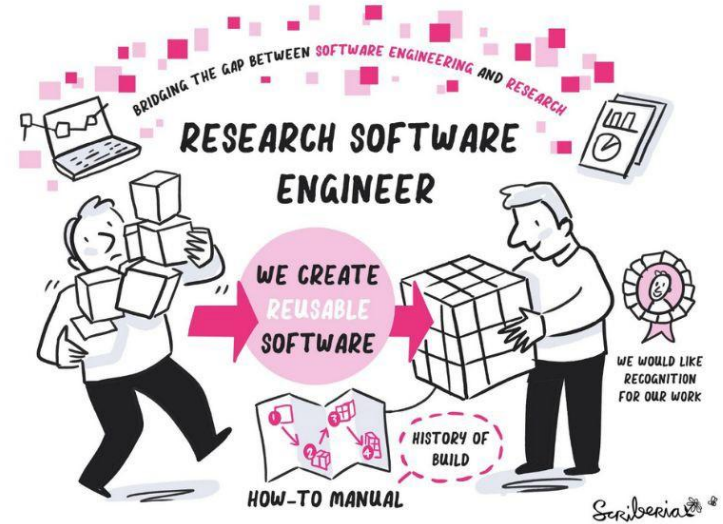
Research

Software Developer

Non-technical complexities of the
research environment

What is Research Software

Foundational algorithms, the software itself, as well as scripts and computational workflows that were created during the research process or for a research purpose, across all domains of research.



Setting up a (research) code project



DIGITAL RESEARCH
ACADEMY

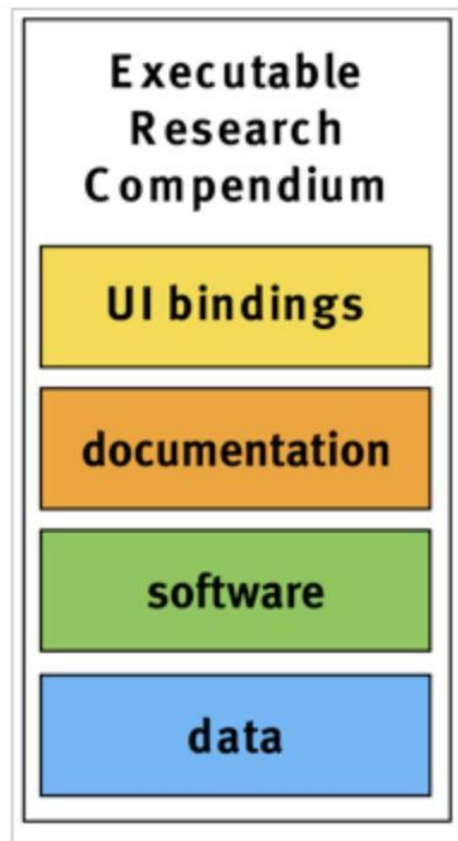
The Research compendium



This image was created by [Scriberia](#) for The Turing Way community and is used under a CC-BY licence (DOI [10.5281/zenodo.3332807](https://doi.org/10.5281/zenodo.3332807)).

The Research Compendium

- A research compendium is a collection of all digital parts of a research project including data, code, texts (protocols, reports, questionnaires, meta data). **The collection is created in such a way that reproducing all results is straightforward.**
- A research compendium accompanies, enhances, or is a scientific publication providing data, code, and documentation for reproducing a scientific workflow.



Nuest, D., Konkol, M., Pebesma, E., Kray, C., Schutzeichel, M., Przibytzin, H., & Lorenz, J. (2017). Opening the Publication Process with Executable Research Compendia. *D-Lib Magazine*, 23(1/2). [10.1045/january2017-nuest](https://doi.org/10.1045/january2017-nuest)

The research compendium/the repro pub

Published July 15, 2019 | Version v1

Conference paper

 Open

The ReproPub: A hybrid research object for supporting publication-level re-execution and generalization of neuroimaging research findings

Kennedy, David N¹ 

Show affiliations

- Idea: create a workflow that re-creates a publication from raw data to pdf

Neurolibre

- FULLY reproducible paper
- Living preprint: reviewer comments are visible; interactive document

A reproducible benchmark of resting-state fMRI denoising strategies using fMRIPrep and Nilearn

Jupyter Notebook Python Submitted 13 May 2023 • Published 19 June 2023



A reproducible benchmark of resting-state fMRI denoising strategies using fMRIPrep and Nilearn

DOI: [10.55458/neurolibre.00012](https://doi.org/10.55458/neurolibre.00012)

Reproducible Preprint

Hao-Ting Wang^{1,7}, Steven L. Meisler^{2,3}, Hanad Sharmarke¹, Natasha Clarke¹, François Paugam⁴, Nicolas Gensollen⁵, Christopher J. Markiewicz⁶, Bertrand Thirion⁵, and Pierre Bellec^{1,7}

 LIVING PREPRINT

 Download PDF

 Preprint repository

 Technical screening

 Repository archive

 Data archive

 HTML archive

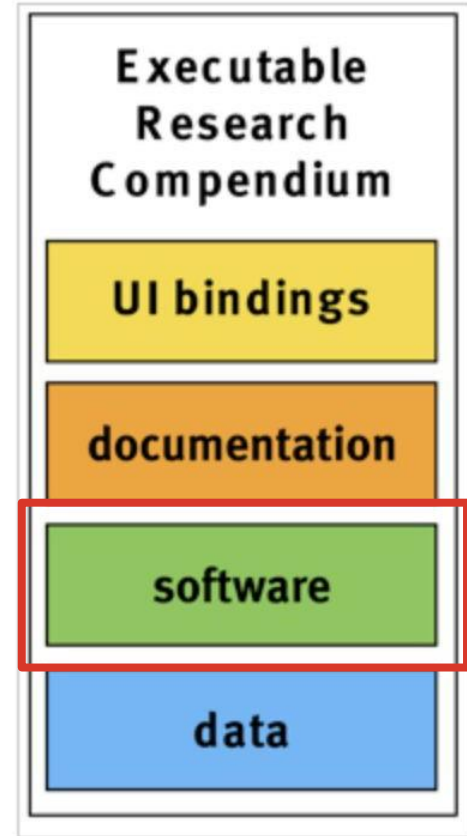
 Container archive

The Research Compendium

A very comprehensive website on research compendia can be found [here](#)

An example template to document your research can be found [here](#)

Reproducible science includes a good organisation of the software/code



Setting up a code project - as part of a research compendium

The folder structure

```
compendium/  
├── data  
│   └── my_data.csv  
├── analysis  
│   └── my_script.R  
├── DESCRIPTION  
└── README.md
```

```
compendium/  
├── CITATION  
├── code  
│   ├── analyse_data.R  
│   └── clean_data.R  
├── data_clean  
│   └── data_clean.csv  
├── data_raw  
│   ├── datapackage.json  
│   └── data_raw.csv  
├── Dockerfile  
├── figures  
│   └── flow_chart.jpeg  
├── LICENSE  
├── Makefile  
├── paper.Rmd  
└── README.md
```

Separate Methods, Data, Output

- **Read-only:** raw data (`data_raw\`), metadata (`datapackage.json`, `CITATION`)
- **Human-generated:** code (`clean_data.R`, `analyse_data.R`), paper (`paper.Rmd`), documentation (`README.md`)
- **Project-generated:** clean data (`data_clean\`), figures (`figures\`), other output

Setting up a code project

```
my-research-project/
├── .vscode/           # vs code settings
│   ├── launch.json    # Cleaned, derived data
│   └── settings.json  # Data documentation
├── src/               # Code (modules, scripts)
├── notebooks/         # Jupyter or MATLAB Live Scripts
├── data/
│   ├── raw/           # Untouched raw data
│   ├── processed/     # Cleaned, derived data
│   └── README.md      # Data documentation
├── results/
│   ├── figures/       # Plots/images generated
│   └── tables/        # CSV, LaTeX tables, etc.
├── paper/
│   ├── manuscript.md  # Or .tex, .docx,
│   ├── bibliography.bib # References
│   └── figures/       # Final figures for the paper
├── tests/             # Automated tests
├── requirements.txt   # or environment.yml / setup.py
├── Makefile           # Automate workflows (optional)
├── LICENSE
├── README.md          # What is your project about?
└── .gitignore
```

Cookiecutter project template

- Code and data version for good file organisation

```
johannabayer@Hanna my_project % tree
.
├── LICENSE
├── Makefile
├── README.md
├── data
│   ├── external
│   ├── interim
│   ├── processed
│   └── raw
├── docs
├── models
├── my_project
│   ├── __init__.py
│   ├── config.py
│   ├── dataset.py
│   ├── features.py
│   ├── modeling
│   │   ├── __init__.py
│   │   ├── predict.py
│   │   └── train.py
│   └── plots.py
├── notebooks
├── pyproject.toml
├── references
├── reports
│   └── figures
├── requirements.txt
└── setup.cfg
```


Project folder structure

- Cookie cutter project templates
- [Cookie cutter data science \(ccds\)](#)
 - Python-based project structure
- Can be a start if you more advanced



Cookiecutter Data Science

Cookiecutter Data Science

Home

Quickstart
Starting a new project
Example
Directory structure

Why ccds?
Opinions
Using the template
All options
Contributing
Related projects
v1 Template

DRIVEN
DATA

Cookiecutter Data Science
is a DrivenData project.

Cookiecutter Data Science

A logical, flexible, and reasonably standardized project structure for doing and sharing data science work.

pypt v2.3.0 python 3.9 | 3.10 | 3.11 | 3.12 | 3.13 CCDS Project template tests failing

CCDS V2 Announcement

Version 2 of Cookiecutter Data Science has launched recently. To learn more about what's different and what's in progress, see the [announcement blog post](#) for more information.

Quickstart

Cookiecutter Data Science v2 requires Python 3.9+. Since this is a cross-project utility application, we recommend installing it with [pipx](#). Installation command options:

With pipx (recommended) With pip With conda (coming soon!) Use the v1 template

```
pipx install cookiecutter-data-science
```

```
# From the parent directory where you want your project  
ccds
```

<https://cookiecutter.readthedocs.io/en/stable/>

Setting up a coding project - naming files

Files need to be

- Human readable
- Machine readable
- Consistent
- Descriptive
- Optional: Play with default ordering (i/e start your file with creation date
YYYY-MM-DD).



Naming files

(BAD) Examples:

- Myabstract.docx
- Joe's Filenames Use Spaces and Punctuation.xlsx
- figure 1.png
- fig 2.png
- JW7d^(2sl@deletethisandyourcareerisoverWx2*.txt

 FINAL_PACE400 with PNS variables v2 (brief PNS dataset_PAS_LatePace_fulldates_CAARMSPACE400_oct2019_original.sav



mergedPACE400 + LatePACE_original.sav

Naming files

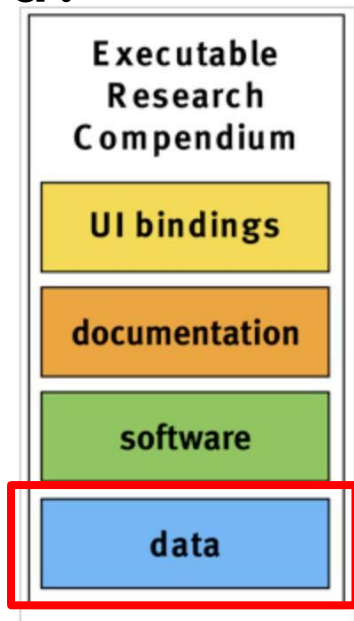
Better
examples

- YYYYMMDD_OHBM_abstract.txt
- Step_1_step_description
- Step_2_step_description
- Unique filenames:
 - Sub_1
 - sub_1_fileA.txt
 - Sub_1_fileB.txt
 - Sub_2-
 - sub_2_fileA.txt
 - Sub_2_fileB.txt

Follow the software example:
versioning

- Generic version is the CURRENT/IMPORTANT version (eg. OHBM_abstratc.txt)
- All older/other versions are versioned (e.g OHBM_abstract_Sept_2024.txt)
- Similar to JAVA/Git - > main method/master is the one that compiles, not main_2025, even if that's the latest version.

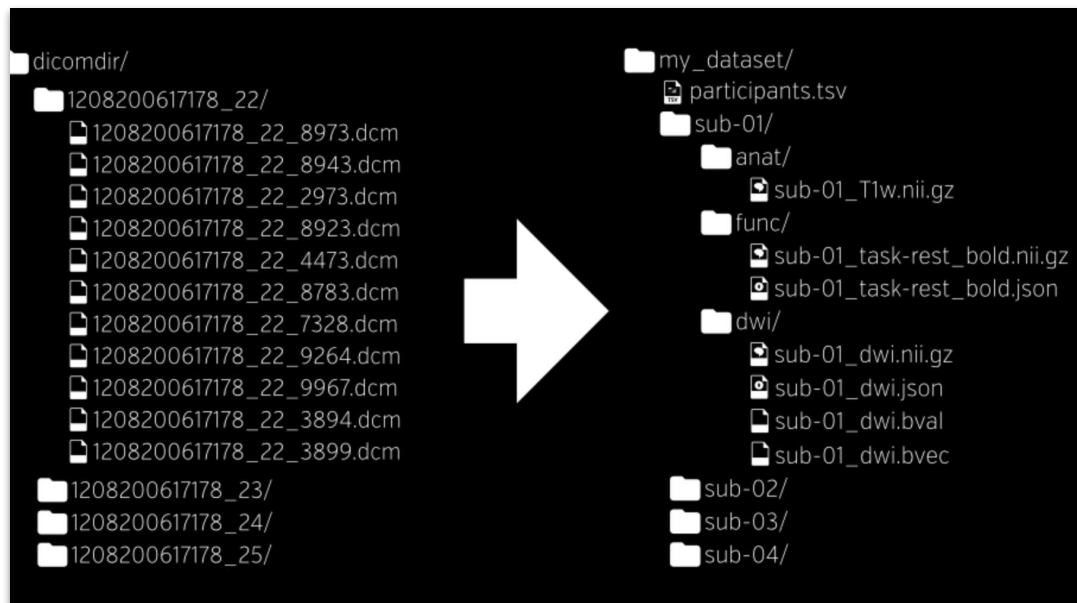
Data organisation - does your field have a file standard?



```
my-research-project/  
├── .vscode/           # vs code settings  
│   ├── launch.json    # Cleaned, derived data  
│   └── settings.json  # Data documentation  
├── src/               # Code (modules, scripts)  
├── notebooks/         # Jupyter or MATLAB Live Scripts  
├── data/              #  
│   ├── raw/           # Untouched raw data  
│   ├── processed/     # Cleaned, derived data  
│   └── README.md      # Data documentation  
├── results/           #  
│   ├── figures/       # Plots/images generated  
│   └── tables/        # CSV, LaTeX tables, etc.  
├── paper/             #  
│   ├── manuscript.md  # Or .tex, .docx, etc.  
│   ├── bibliography.bib # References  
│   └── figures/       # Final figures for the paper  
├── tests/             # Automated tests  
├── requirements.txt    # or environment.yml / setup.py  
├── Makefile           # Automate workflows (optional)  
├── LICENSE  
├── README.md          # What is your project about?  
└── .gitignore
```

Does your field have a file standard?

Brain Imaging Derived Structures



(<https://bids.neuroimaging.io/>):

Slides by courtesy of Dr. Remi Gau, adapted

File organisation: (stealing from the) BIDS standard

Brain Imaging Derived Structures

- modularizes data
- names files in a human AND machine friendly way
- specifies a folder structure
- uses standard interoperable file formats
- documents metadata
- minimize duplication



File organisation: (stealing from the) BIDS standard

Brain Imaging Derived Structures

Key1-value_key2-value_suffix.extension

sub-001_population-cross_mri.nii

- Suffix preceded by an underscore
- Entities are composed of key-label pairs separated by underscores
- Key and label separated by hyphen
- Keys, labels, suffixes can only contain letters and / or numbers.
- Key-value combination can be repetitions from higher order folders to form unique filenames

Adding a Readme.md

- Entry point/first item that visitor reads
- Follows Markdown format
- There are several (slightly differing) markdown formats, but we are going to focus on [git markdown](#).

```
my-research-project/  
├── .vscode/           # vs code settings  
│   ├── launch.json    # Cleaned, derived data  
│   └── settings.json  # Data documentation  
├── src/               # Code (modules, scripts)  
├── notebooks/         # Jupyter or MATLAB Live Scripts  
├── data/                
│   ├── raw/           # Untouched raw data  
│   ├── processed/     # Cleaned, derived data  
│   └── README.md      # Data documentation  
├── results/             
│   ├── figures/       # Plots/images generated  
│   └── tables/        # CSV, LaTeX tables, etc.  
├── paper/               
│   ├── manuscript.md  # Or .tex, .docx, etc.  
│   ├── bibliography.bib # References  
│   └── figures/       # Final figures for the paper  
├── tests/             # Automated tests  
├── environment.yml     # or requirements.txt / setup.py  
├── Makefile            # Automate workflows (optional)  
├── LICENSE  
├── README.md           # What is your project about?  
└── .gitignore
```

Readme.md

Files ending in .md indicate the Markdown style. We are going to follow [Github Markdown](#).

A Readme.md should contain:

- Title and description of the project
- Author of the project and contact details
- Information about the license of the project - can everyone reuse your code?
- Structure of the project
- Anything that a user of the project should know

Examples: [pydantic](#)



Thinking about the writing early on

```
my-research-project/
├── .vscode/           # vs code settings
├── launch.json        # Cleaned, derived data
├── settings.json      # Data documentation
├── src/              # Code (modules, scripts)
├── notebooks/        # Jupyter or MATLAB Live Scripts
├── data/
│   ├── raw/          # Untouched raw data
│   ├── processed/    # Cleaned, derived data
│   └── README.md      # Data documentation
├── results/
│   ├── figures/      # Plots/images generated
│   └── tables/       # CSV, LaTeX tables, etc.
├── paper/
│   ├── manuscript.md # Or .tex, .docx, etc.
│   ├── bibliography.bib # References
│   └── figures/      # Final figures for the paper
├── tests/            # Automated tests
├── environment.yml   # or requirements.txt / setup.py
├── Makefile          # Automate workflows (optional)
├── LICENSE
├── README.md         # What is your project about?
└── .gitignore
```

The repro pub - text editor and reference management system (based on reproducibility criteria)

Text editor	pro	con
MS Word/Google docs	-Plugins for most reference managers -good (?) platform for getting feedback/review -(Version controlled)	- Figures need to be imported each time
Markdown	-Integration with R (markdown, knitr) -In theory, fully reproducible (code and paper)	- No review option
Overleaf (online)	-Bibtex/Latex -Pro version has direct git and Dropbox link -Math support, templates -review option	- steeper learning curve
LaTeX (Desktop)	-Free -Bibtex file -Entirely version controllable -Math support -Create a figure folder that you save your figs in	-Very steep learning curve -No review option

Collaborative publishing using overleaf



- All benefits of pure latex but
 - Working on a document together & review mode (suggest changes)
 - Import citations from paperpile, zotero, mendeley and endnote
 - Templates for publications, journals, conference posters etc
 - Link an image folder (i.e in Dropbox)
 - Git integration

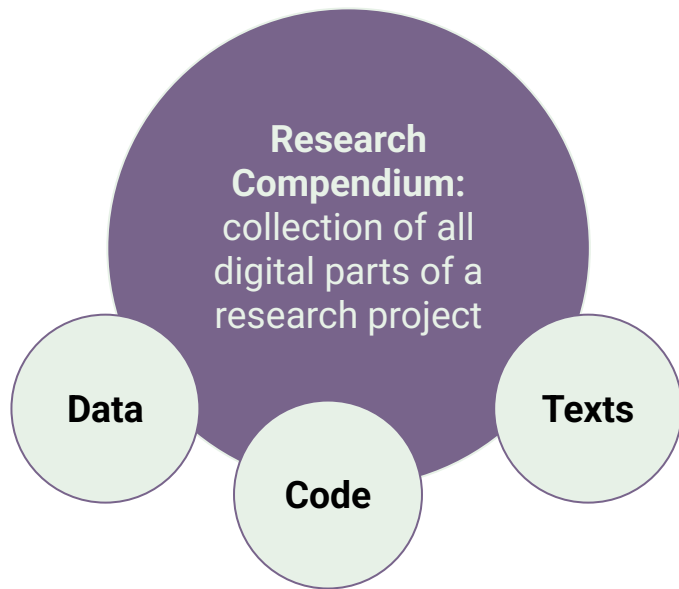
Downside

- Internet dependency (except offline mode)

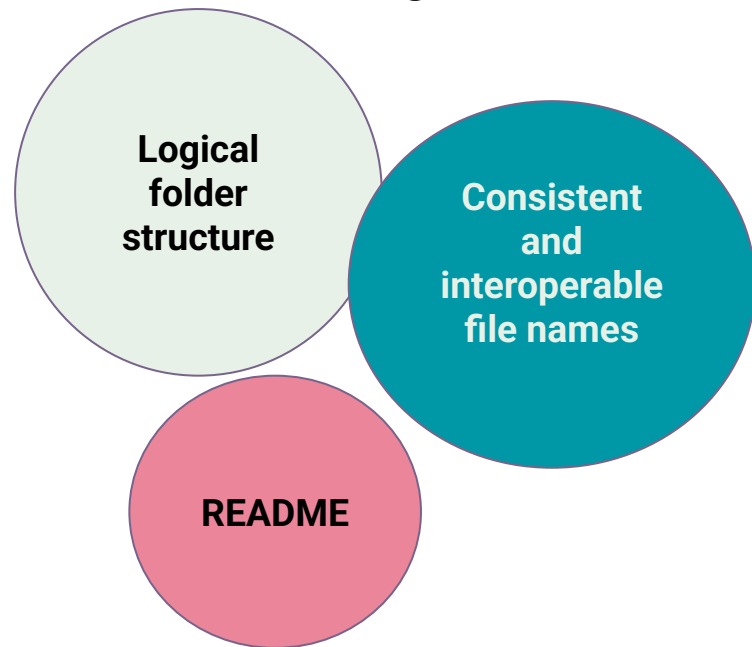
More resources on Research Compendia

- <https://github.com/topics/research-compendium/>
- <https://the-turing-way.netlify.app/reproducible-research/compendia>
- <https://research-compendium.science/>
- [https://selinazitrone.github.io/YoMos2020/s_lidy_presentation.html#\(1\)](https://selinazitrone.github.io/YoMos2020/s_lidy_presentation.html#(1))

Summary



The collection is created in such a way that reproducing all results is straightforward



Break

(Software) version control



DIGITAL RESEARCH
ACADEMY

Version control

Git and Github

GitHub is a web-based platform designed for version control, collaboration, and project management. It uses **Git**, a distributed version control system, to enable developers, researchers, and teams to track changes in code, manage projects, and collaborate on software and other digital projects.

GitHub is widely used in software development, open-source projects, research, and even non-code workflows due to its robust collaboration tools and integration capabilities.

Githubs main purpose:

- Version control
- Collaboration

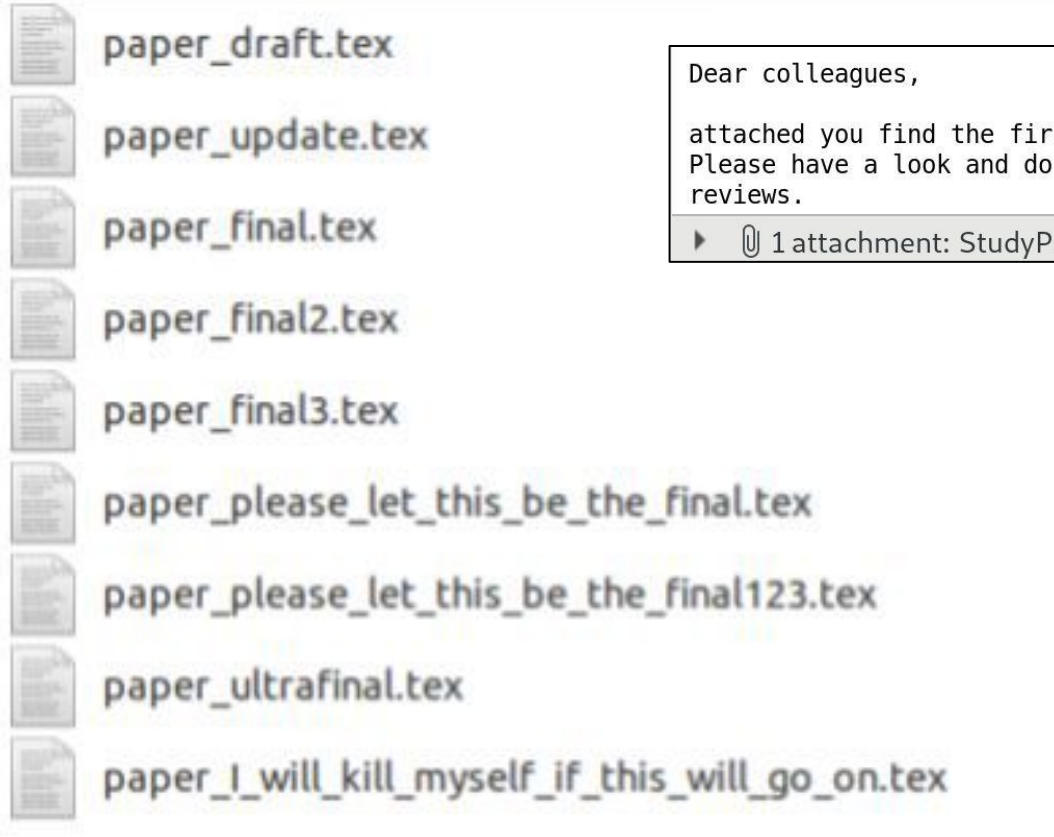


Version control using Github/Gitlab



Community, T. T. W., & Scriberia. (2020). *Illustrations from the Turing Way book dashes*. Zenodo. [10.5281/ZENODO.3695300](https://doi.org/10.5281/ZENODO.3695300)

Version Control ... not!



Dear colleagues,

attached you find the first public version of the [REDACTED] protocol. Please have a look and do comment. We can also meet to aggregate our reviews.

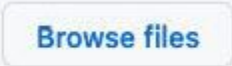
▶ 1 attachment: StudyProposal[REDACTED]_Validation_V1_250918docx.docx

Version Control: Git as a solution

 **Improve grammar in compendia.md (#1822)**

 **main (#1822)**

 **v1.0.3** ... **v1.0.0**



 **inwaves** and **malvikasharan** committed on Apr 24, 2021 Verified

1 parent 328d54a commit 72039372a2803a4ec15764b2c01017039320f096

132	- In the future, the research compendium may even be the publication itself which is being peer reviewed (rather than just peer reviewing the paper, why not review the entire research project).
133	-
133	+ In the future, the research compendium may even be the publication itself allowing peer review of the entire research project.

Screenshot:

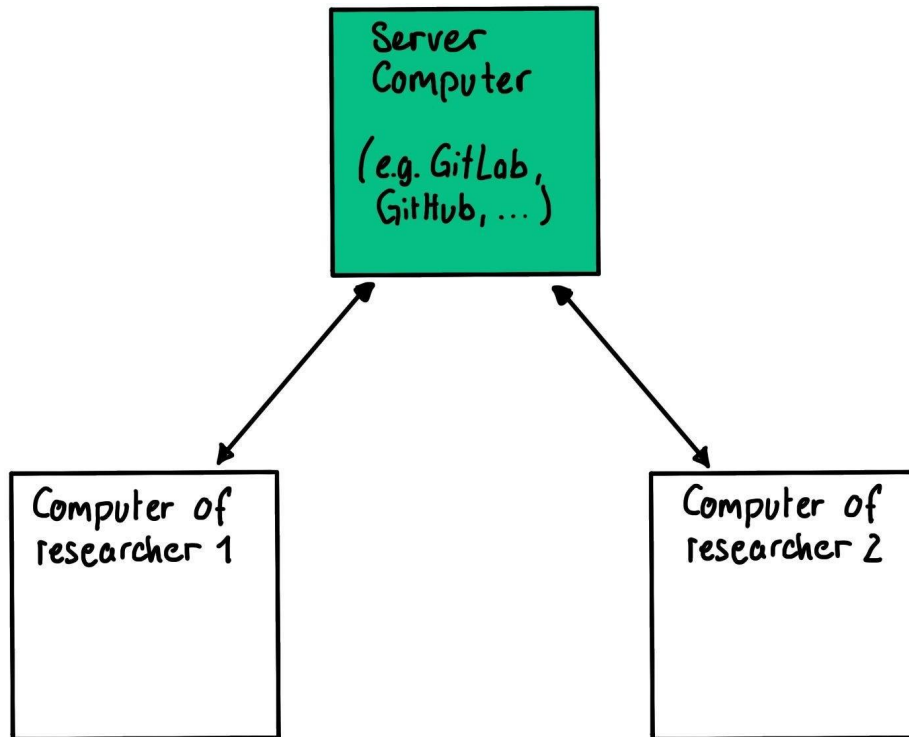
<https://github.com/alan-turing-institute/the-turing-way/co/commit/72039372a2803a4ec15764b2c01017039320f0>

How does Version Control work?

*Track **who** made **which** change and **when***

- Track different versions of your code, text, ...
- Description of each version (“commit message”)

Git



Git

t

Software on your
machine



GitLab

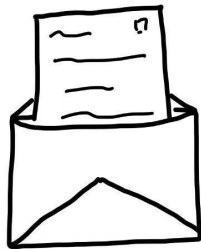
Service to use as remote
server



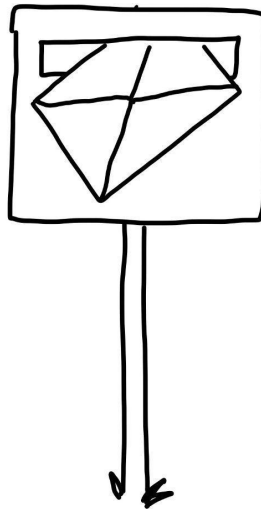
Git **commands** for creating changes



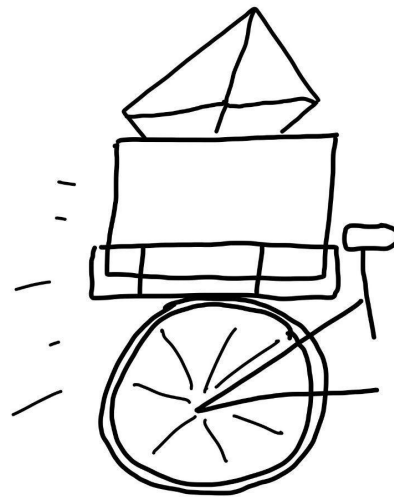
edit
document



add



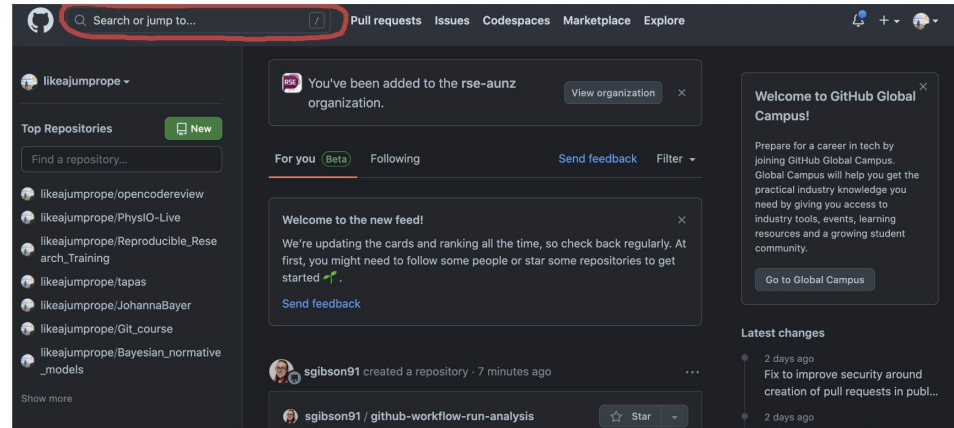
commit



push

Getting to know the Github interface

1. Log into your github account
2. The login page will get you to your dashboard.
Here you find:
 - a. An overview of your dashboard
 - b. Your profile
 - c. An overview of your repositories

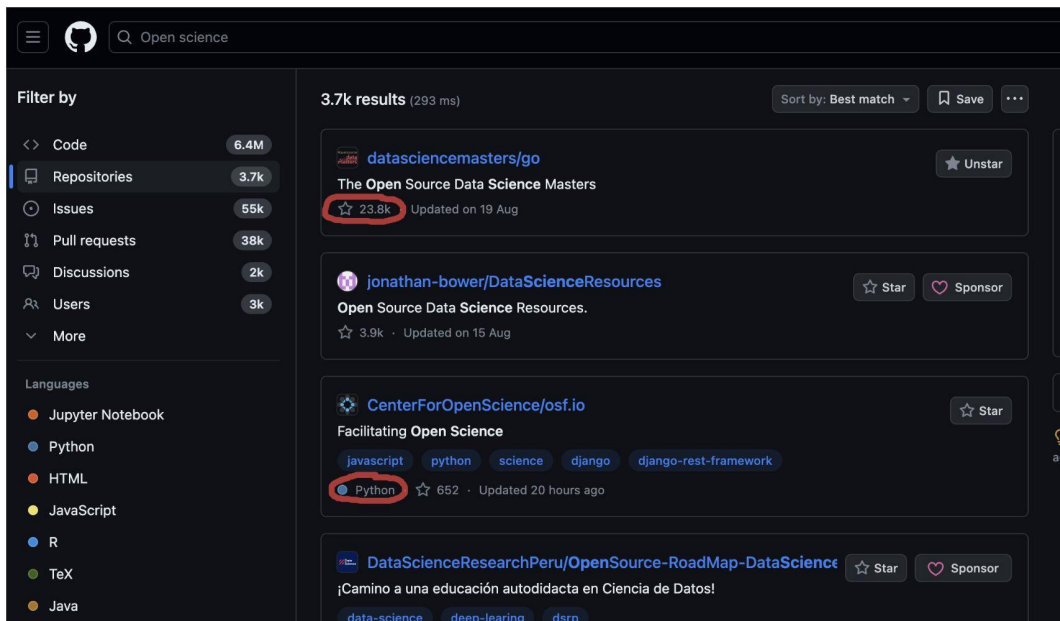


Getting to know the Github interface

Repositories on Github are divided into:

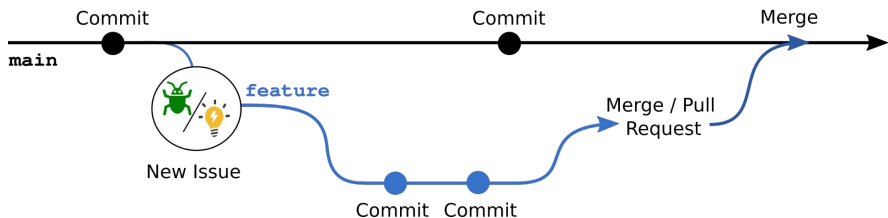
- Those that you have write access to
- Those you can only view (the great majority)

You can use the search function to query all public repositories on Github



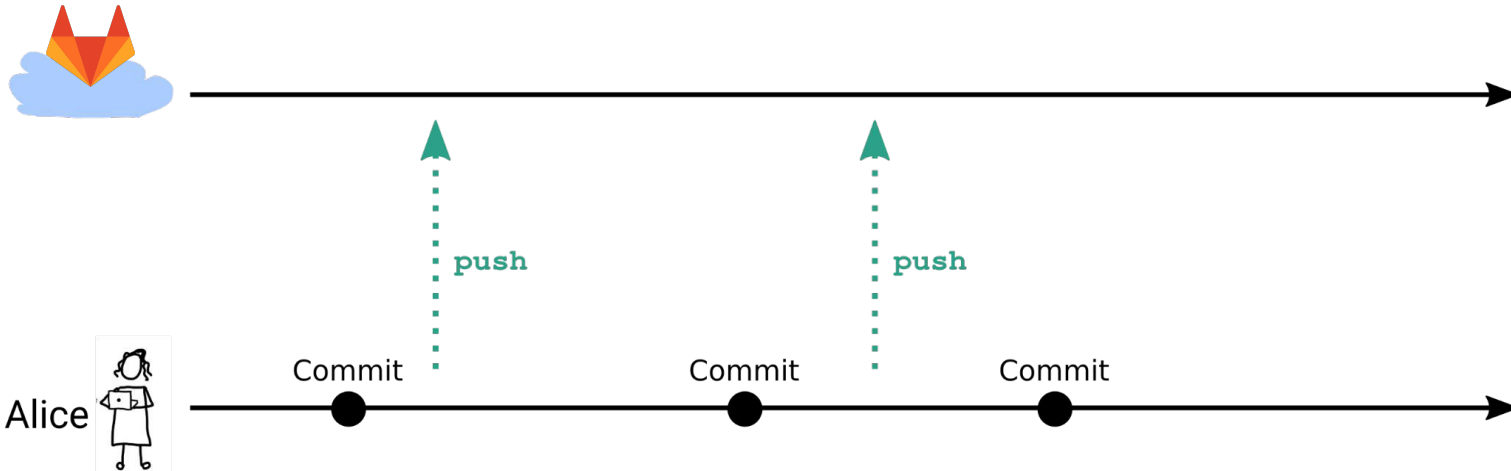
Collaboration with Git

Agenda

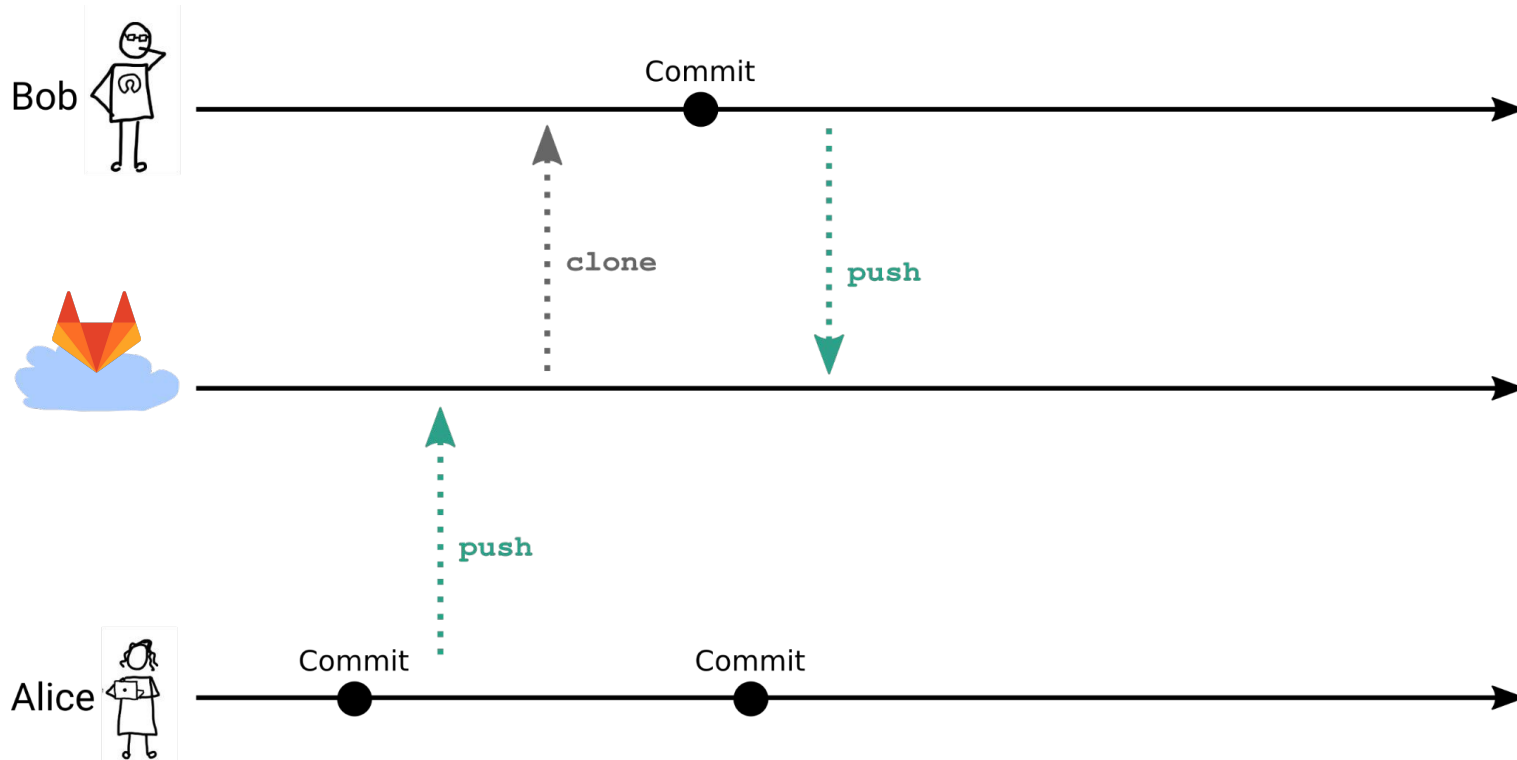


- Integrating changes from remote repository
- Learn GitLab's collaborative features
 - Manage repository access
 - How to create an Issue
 - Create a branch
- Merge branches ...
 - using the command line
 - using GitLab

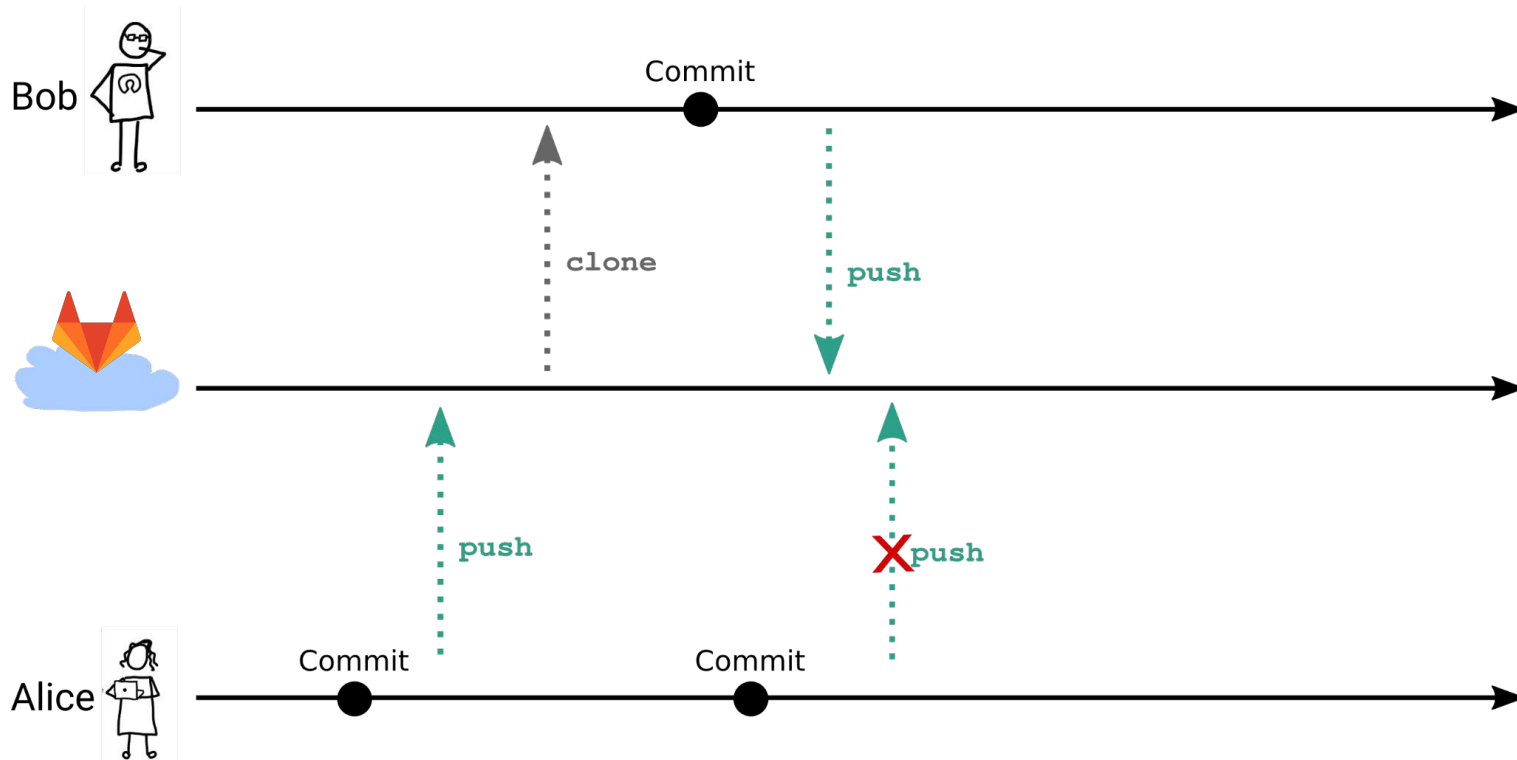
Collaborative Git workflow



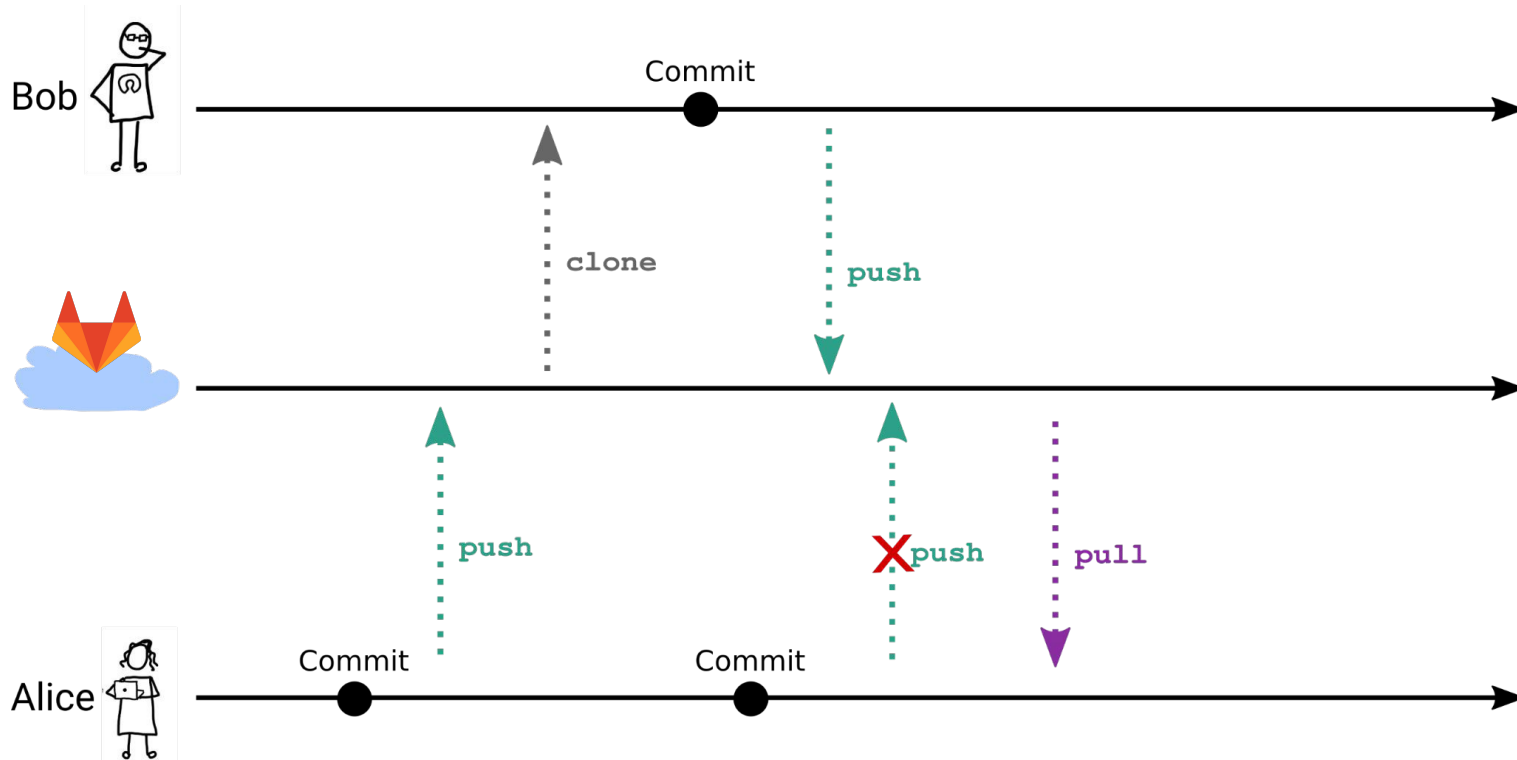
Collaborative Git workflow



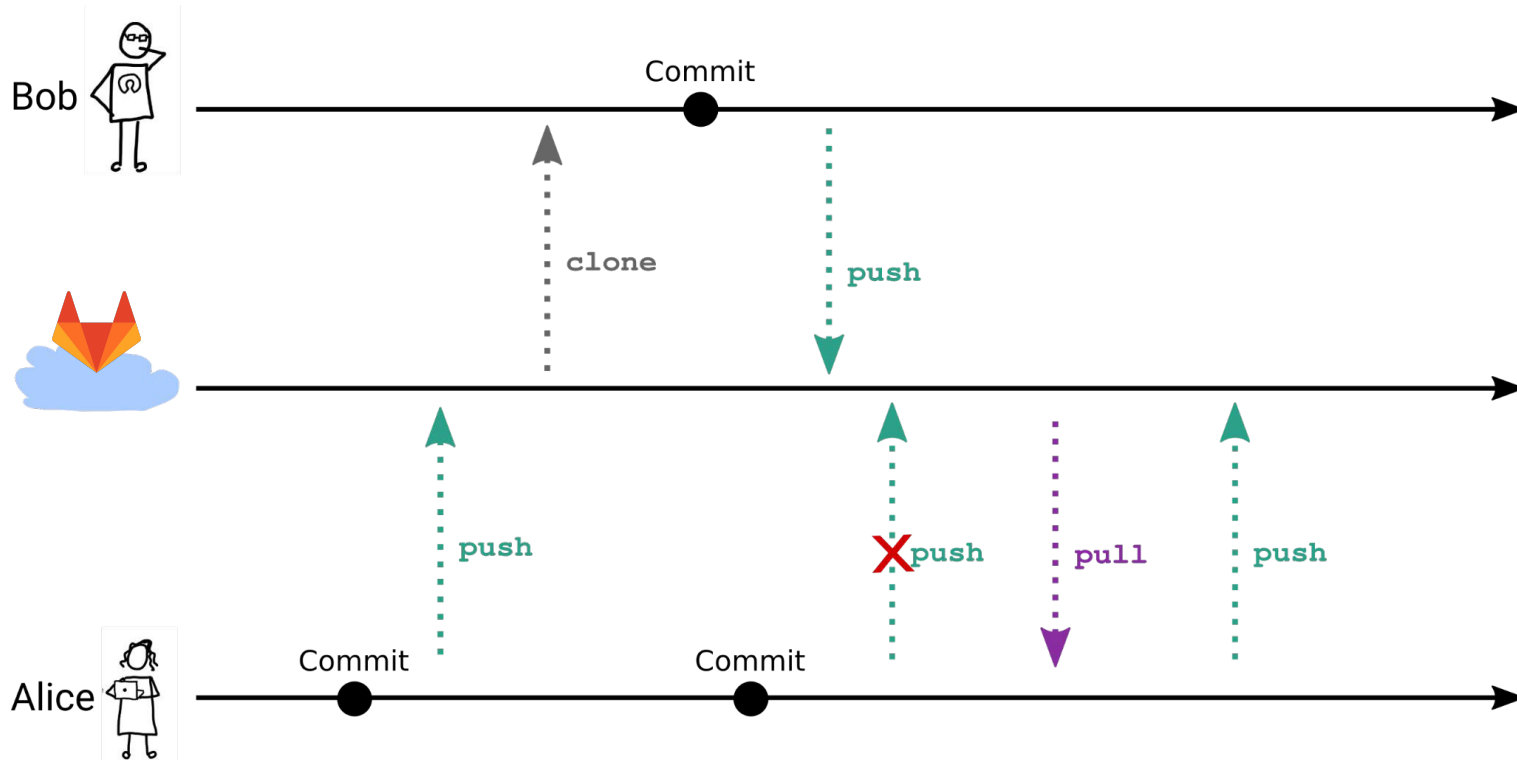
Collaborative Git workflow



Collaborative Git workflow



Collaborative Git workflow



Activity 1 (20')

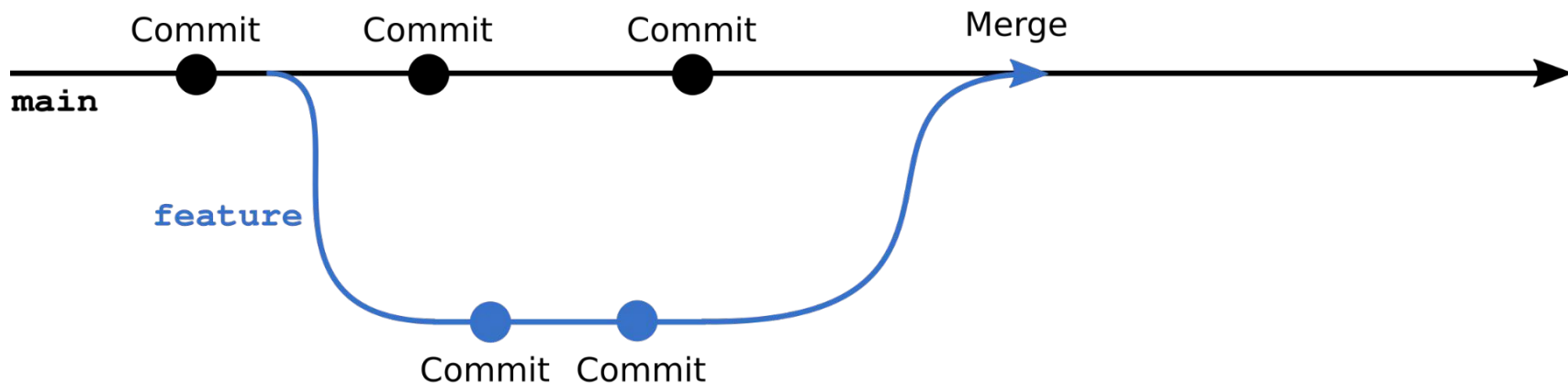
1.

Working on Github online

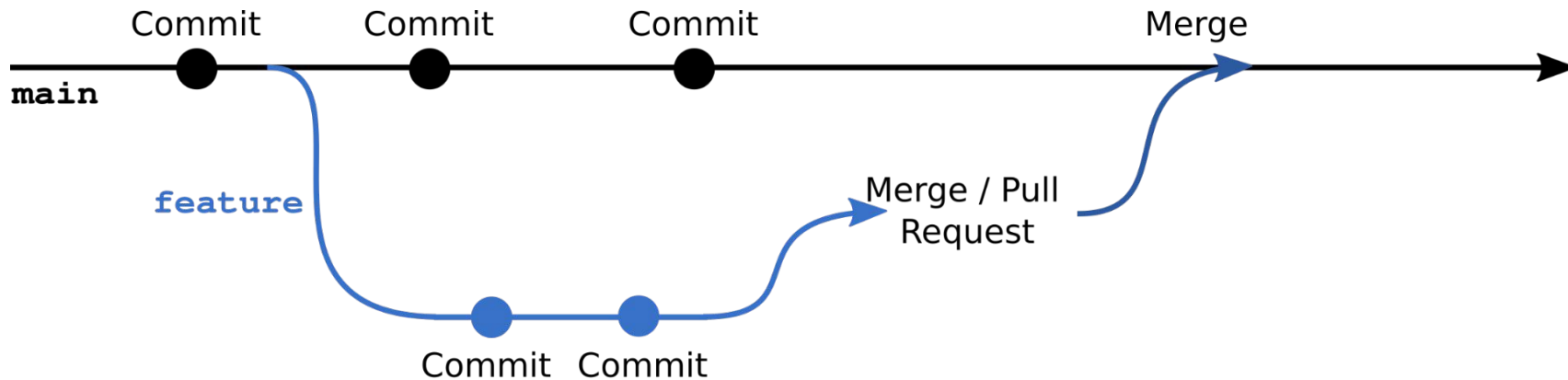
1. Log into your Github account.
2. Fork the course repo:
https://github.com/fritjoflammers/RSE_Juelich
3. In the folder for day1.1, you'll find a Readme.md.
4. Do Activity 1:
 - a. Create a file, add your favourite animal and make a PR.

Branches

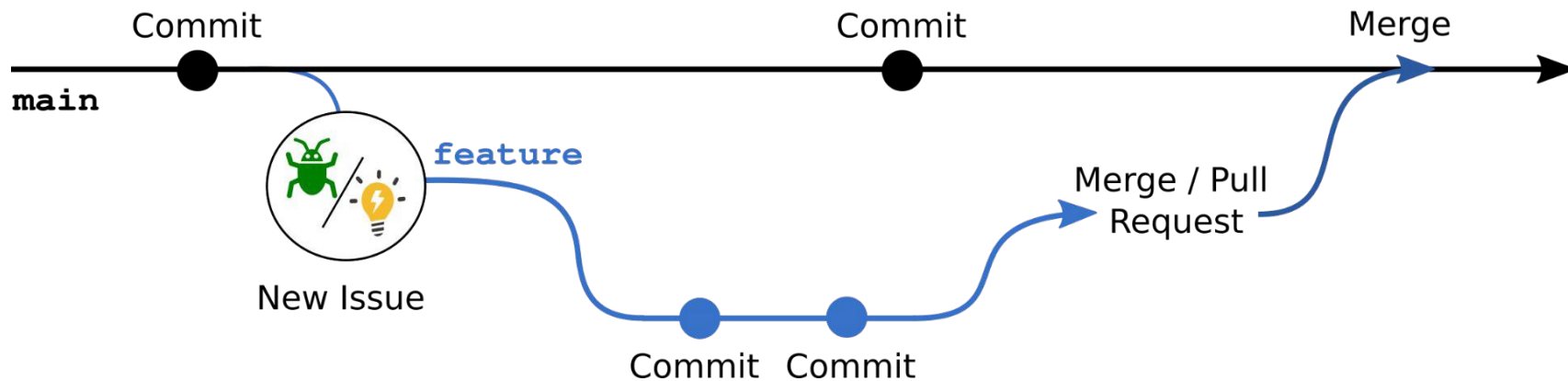
The main-feature branch workflow



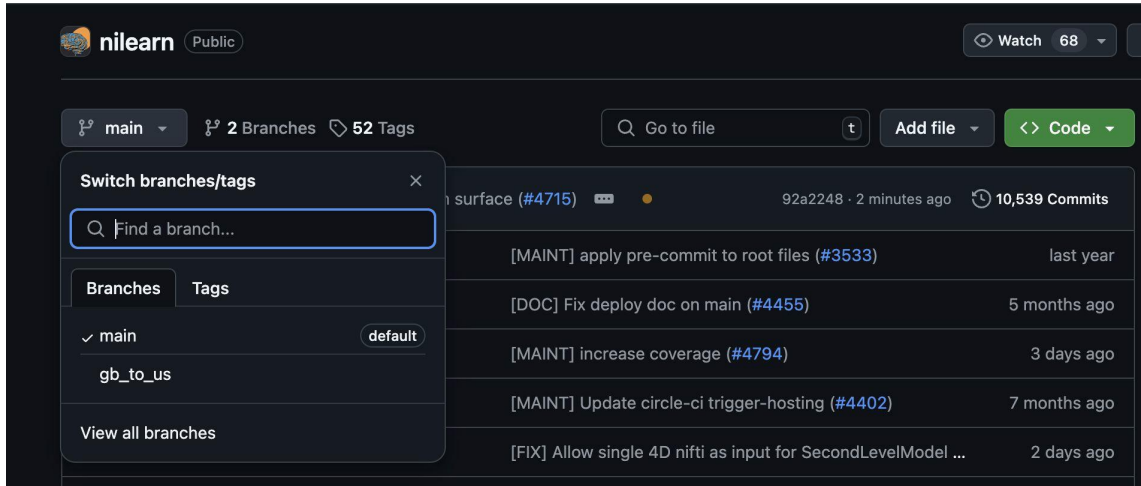
The main-feature branch workflow



A typical collaboration workflow



Make branch and merge back into your code



Activity 2 (15')

Branching

1. Under the Readme.md section of day1.1, do Activity 2:
Create a branch, make a change, merge it back into main

Summary: Recap terms

- **Git**
- **Github / Gitlab**
- **Commits**
- **Forks**
- **Branches**
- **Merge**

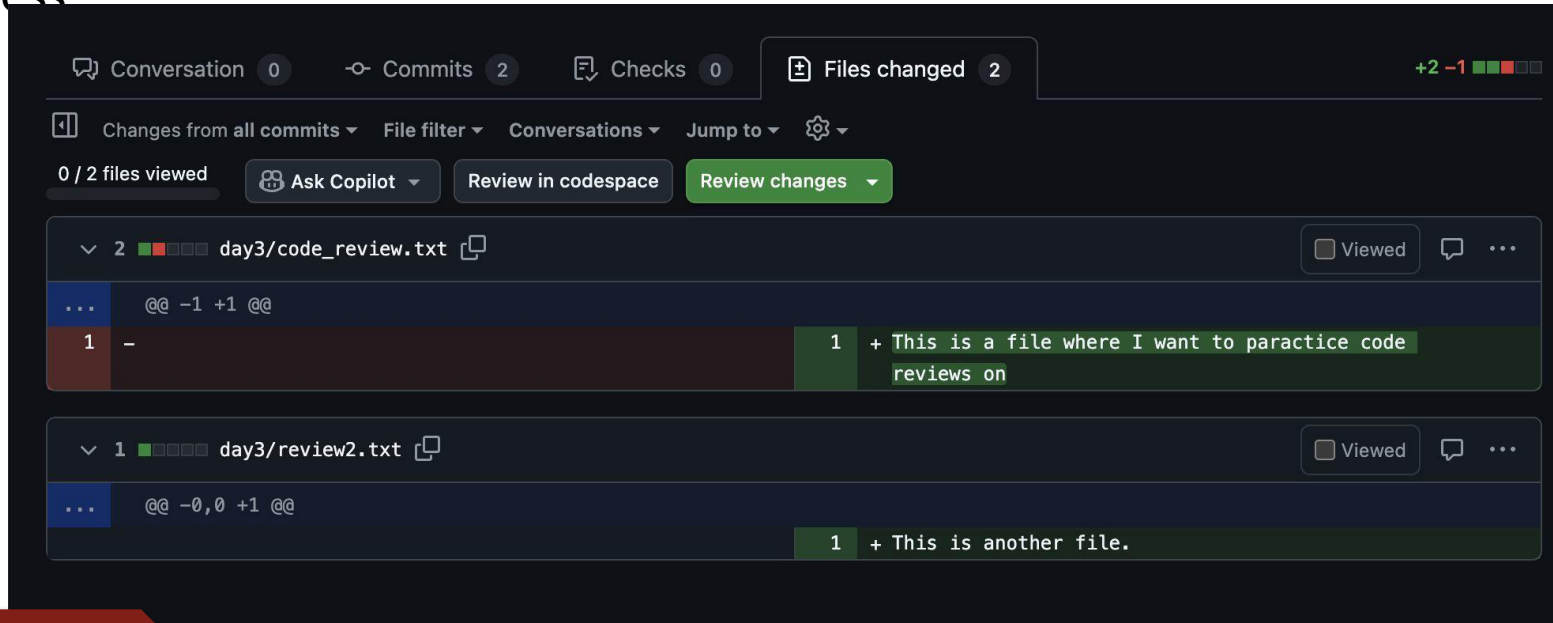
Summary

- **Git** is software providing decentralized version control system
- **Github / Gitlab** are web platforms that provide centralized access to repository and a graphical user interface
- **Commits** are the units of work that contain changes to the repository
- **Forks** are a copy of Github repository in your own account. They make a public, but read-only repository editable for you.
- **Branches** are diversions in the series of commits, allowing parallel work on the same codebase
- **Merge** integrates the changes of two branches into one

Break (15 min)

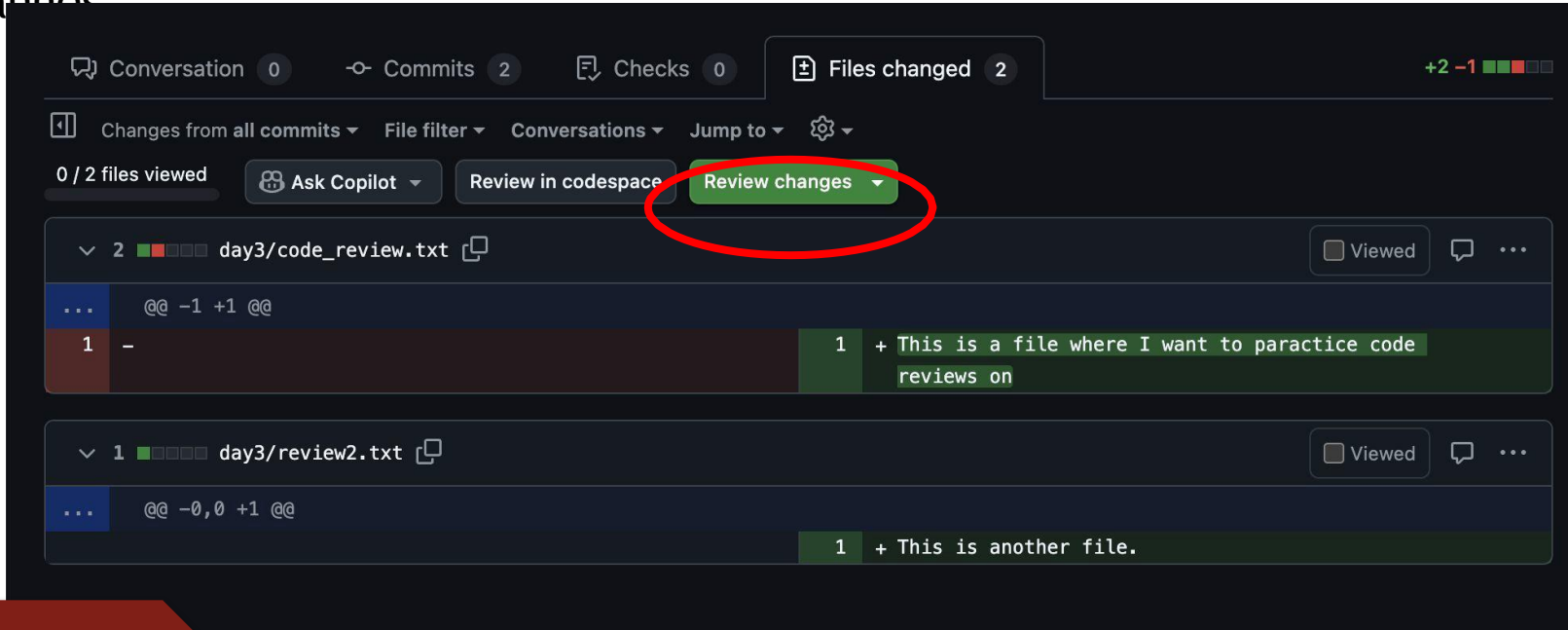
Review a PR online

Github allows for a very efficient code review process



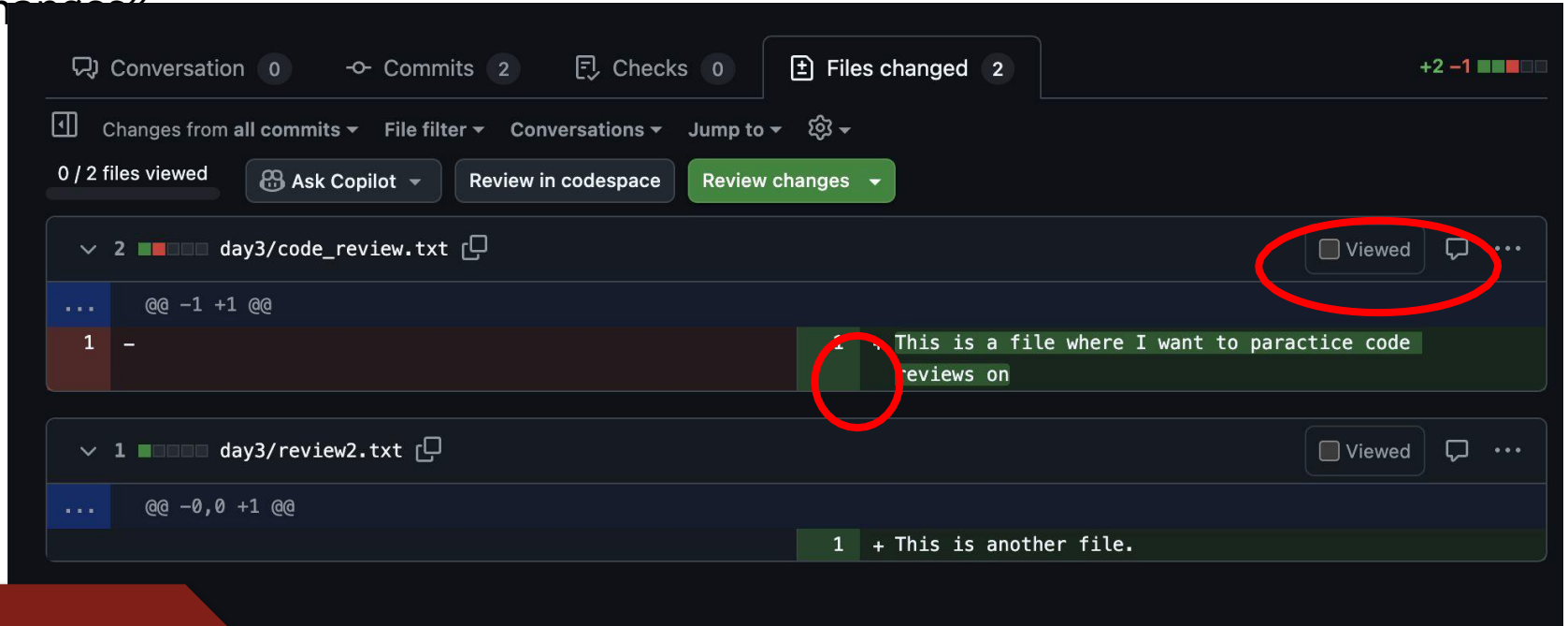
Review a PR online

Click on “review changes”



Review a PR online

Click on “review changes”



Activity 3

(15')

Forking, PR's and code review

Under the Readme.md section of day1, do Activity 3

- Fork each other's repo, make a PR, assign each other as reviewers and perform a code review

?

?

?

What questions do you have?

?



Summary

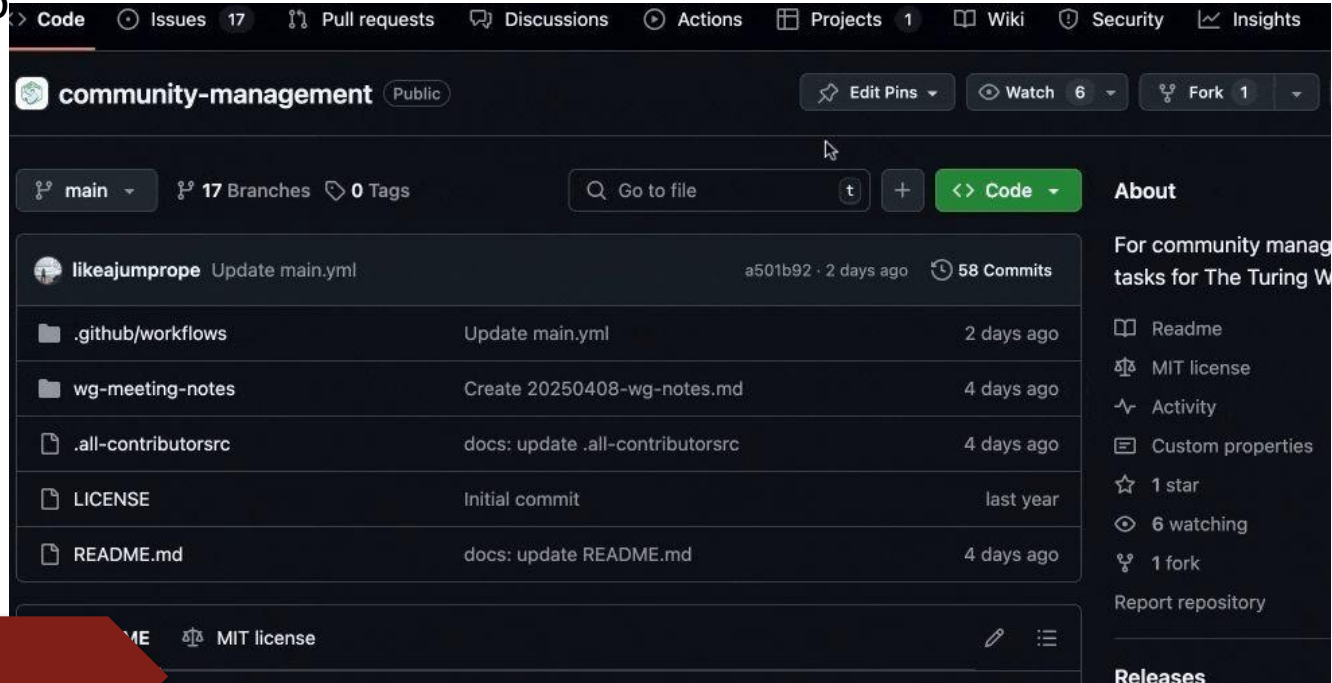
- **Git** is software providing decentralized version control system
- **Github / Gitlab** are web platforms that provide centralized access to repository and a graphical user interface
- **Commits** are the units of work that contain changes to the repository
- **Forks** are a copy of Github repository in your own account. They make a public, but read-only repository editable for you.
- **Branches** are diversions in the series of commits, allowing parallel work on the same codebase
- **Merge** integrates the changes of two branches into one
- **Pull requests** are a Github feature to propose changes, facilitate code review and merge changes into the codebase.

Branch - Commit - Push - Pull request - Review - Merge

Some more
Github
features

Project management - boards on github

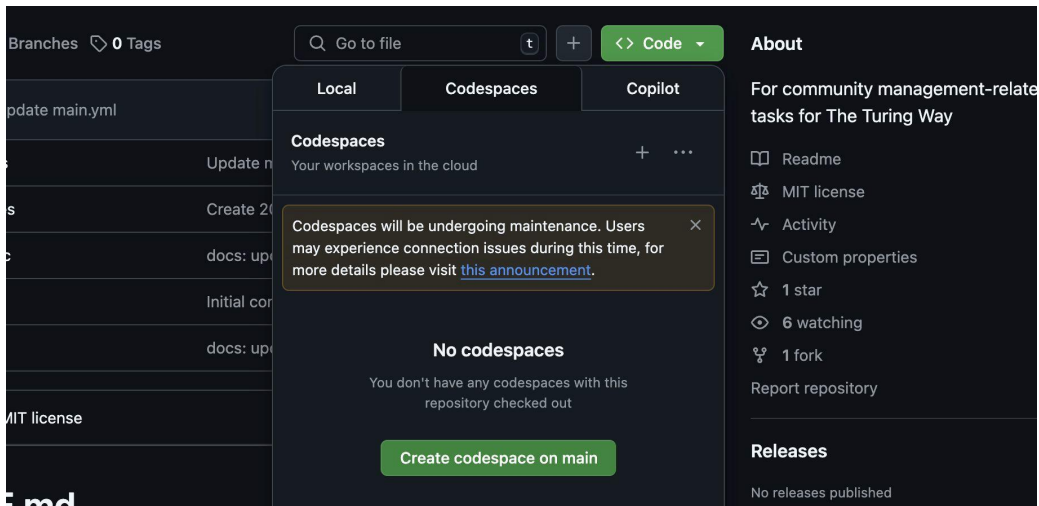
Project boards are a great way to organize a project and issues on github



Codespaces

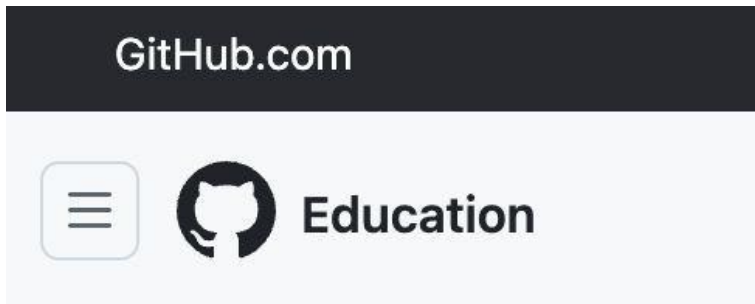
Github codespaces allows you to run files in Github repository in VS code instance in your browser.

- Good for quickly checking things
- CAREFUL: You have some free quota (as student etc), but then it's a paid service
- Close your instance!!



Github Education

- As a registered student, you can gain access to Github education
 - Free perks, such as:
 - Free Co-Pilot and Codespace
 - Full Notion AI plan
 - Licenses to JetBrains IDE etc
 - Certifications



Github Copilot

- Chat-GPT for CODE
- Integratable with VS code
- If you are a registered maintainer of an open source repository, you also get free access to codespaces and co-pilot.



Ressources

- Baxter, R, Chue Hong, N, Gorissen, D, Hetherington, J & Todorov, I 2012, 'The Research Software Engineer', Digital Research 2012, Oxford, United Kingdom, 10/09/12 - 12/09/12.
- The Missing Semester of Your CS Education: <https://missing.csail.mit.edu/>
- Nuest, D., Konkol, M., Pebesma, E., Kray, C., Schutzeichel, M., Przibytzin, H., & Lorenz, J. (2017). Opening the Publication Process with Executable Research Compendia. D-Lib Magazine, 23(1/2). 10.1045/january2017-nuest
- BIDS standard: <https://bids.neuroimaging.io/index.htm>
- Goth, F. et al. Foundational competencies and responsibilities of a Research Software Engineer. F1000Res. 13, 1429 (2024).
- <https://book.the-turing-way.org/collaboration/github-novice>
- <https://book.the-turing-way.org/reproducible-research/vcs>
- https://fritjoflammers.github.io/dra-docker-training/learning-git/02-git_collaboration.html

End

Your code editor

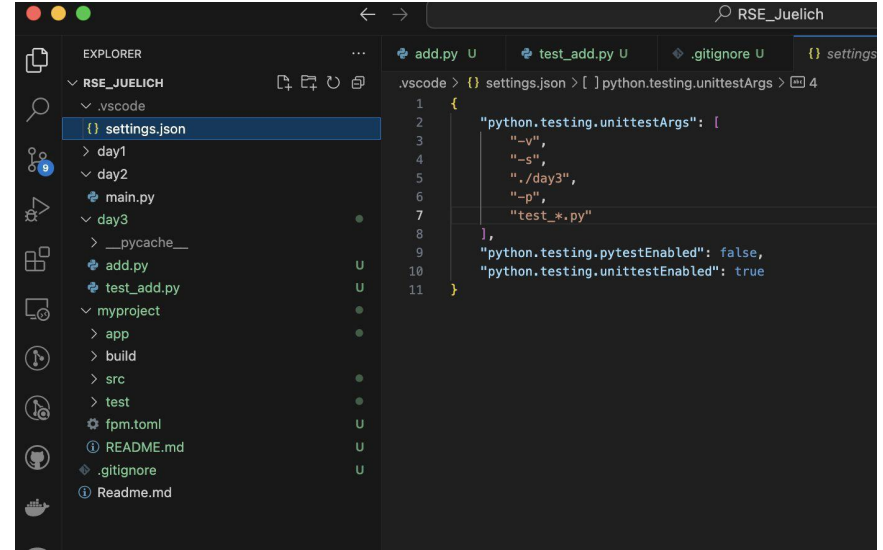
```
my-research-project/
├── .vscode/           # vs code settings
├── launch.json        # Cleaned, derived data
├── settings.json      # Data documentation
├── src/               # Code (modules, scripts)
├── notebooks/         # Jupyter or MATLAB Live Scripts
├── data/
│   ├── raw/           # Untouched raw data
│   ├── processed/     # Cleaned, derived data
│   └── README.md      # Data documentation
├── results/
│   ├── figures/       # Plots/images generated
│   └── tables/        # CSV, LaTeX tables, etc.
├── paper/
│   ├── manuscript.md  # Or .tex, .docx, etc.
│   ├── bibliography.bib # References
│   └── figures/       # Final figures for the paper
├── tests/             # Automated tests
├── environment.yml    # or requirements.txt / setup.py
├── Makefile           # Automate workflows (optional)
├── LICENSE
├── README.md         # What is your project about?
└── .gitignore
```

Setting up your code editor

- VS code
 - Get the extensions for you code base

Important files:

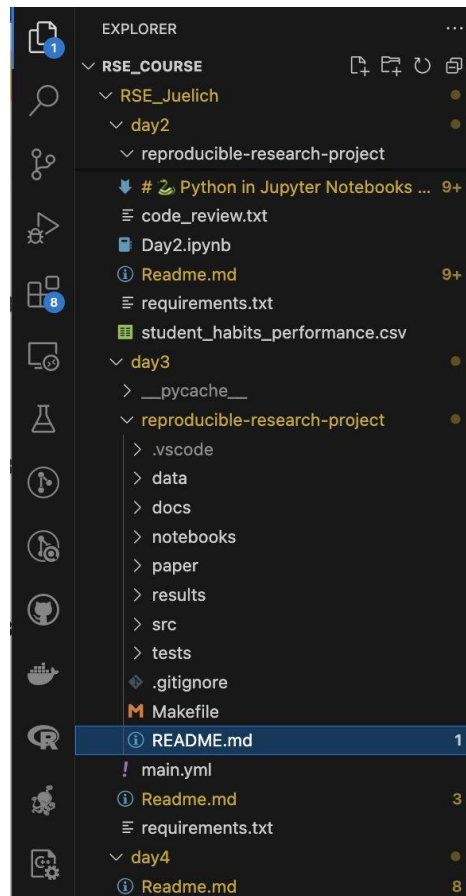
- Settings.json
 - Sets the behaviour of your vscode (linting, formatting etc)
 - Launch.json
 - Specifies the behaviour of vs code running you programs
- Debugging, which interpreter, which language



Important VS code extensions

Extensions for your programming language;

- Python, Julia, etc IDE.
- GitLens
- (Docker)
- Jupyter notebook extension



settings.json

Purpose: Configure editor behavior, extensions, and general environment settings

Scope: User-wide or workspace-specific

Examples:

- Tab size, font, auto-save, linting rules
- Default Python interpreter or Fortran compiler path
- Format-on-save, excluded files, theme

```
elich > .vscode > {} settings.json > ...  
{  
  "python.testing.unittestArgs": [  
    "-v",  
    "-s",  
    "./day3",  
    "-p",  
    "test_*.py"  
  ],  
  "python.testing.pytestEnabled": false,  
  "python.testing.unittestEnabled": true,  
  "python.linting.enabled": true,  
  "editor.tabSize": 4,  
  "files.exclude": {  
    "**/__pycache__": true  
  },  
  "editor.formatOnSave": true  
}
```

launch.json

Specifies:

- Which script or executable to run
- What **command-line arguments** to pass
- Whether to **launch** a new process or **attach** to an existing one
- Where to show the output (terminal or debug console)
- Environment variables, working directory, debugging

ur

```
uelich > .vscode > {} launch.json > ...  
{  
  "name": "Run analysis script",  
  "type": "python",  
  "request": "launch",  
  "program": "${workspaceFolder}/day3/analysis.py",  
  "console": "integratedTerminal",  
  "args": ["--input", "data.csv", "--verbose"]  
}
```

Jupyter notebooks

- Can be installed via ``pip install notebook`` and the launched via ``jupyter notebook notebook.ipynb`` or ``jupyter notebook``
- Allows to run notebooks in the browser
- Not super great for large research projects, but:
 - Operating system agnostic
 - Can be shared and opened in the browser via Google Colab
 - Can be run via vscode
 - A variety of kernels available

For research:

- Better: <https://marimo.io/>



Matlab environment

Matlab projects

Use MATLAB Projects to manage files, paths, and settings.

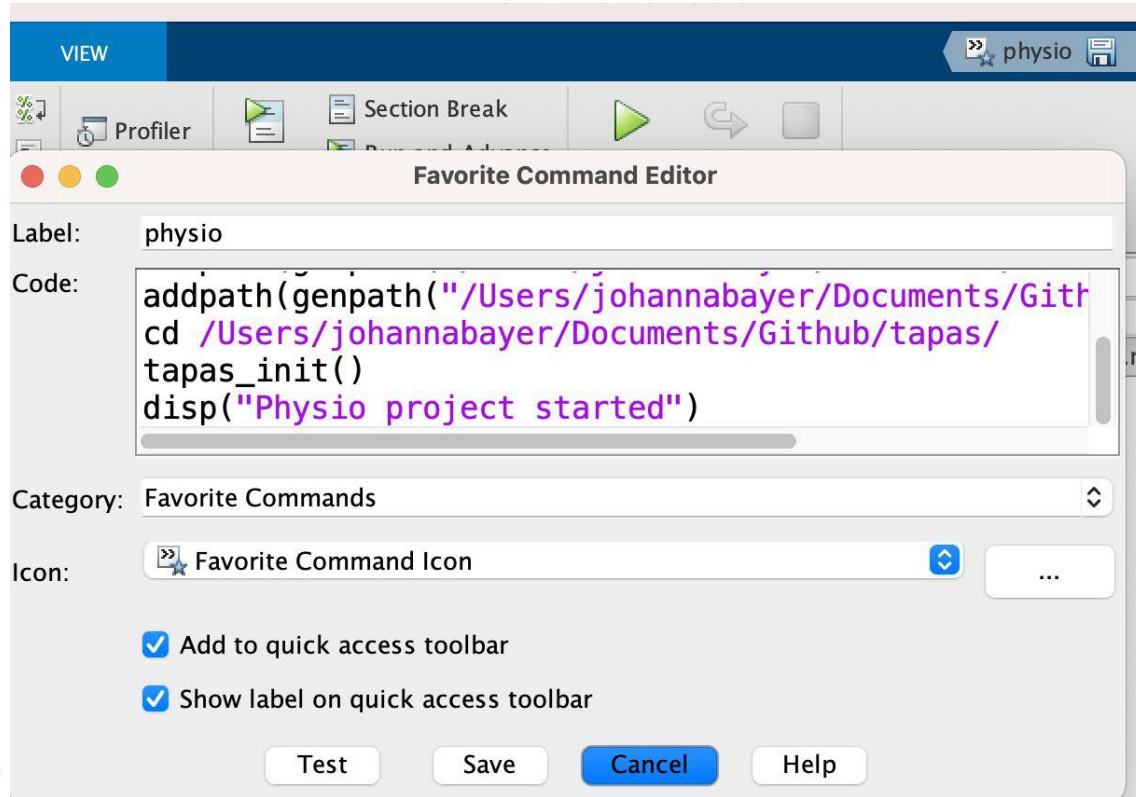
- Go to **Home > New > Project > From Folder**
- Choose your folder (e.g., your project-name/ directory)
- MATLAB creates a .prj file to manage paths, startup code, etc.
-

Configure startup paths:

- **Add a startup.m file** to automatically configure your environment on project open:

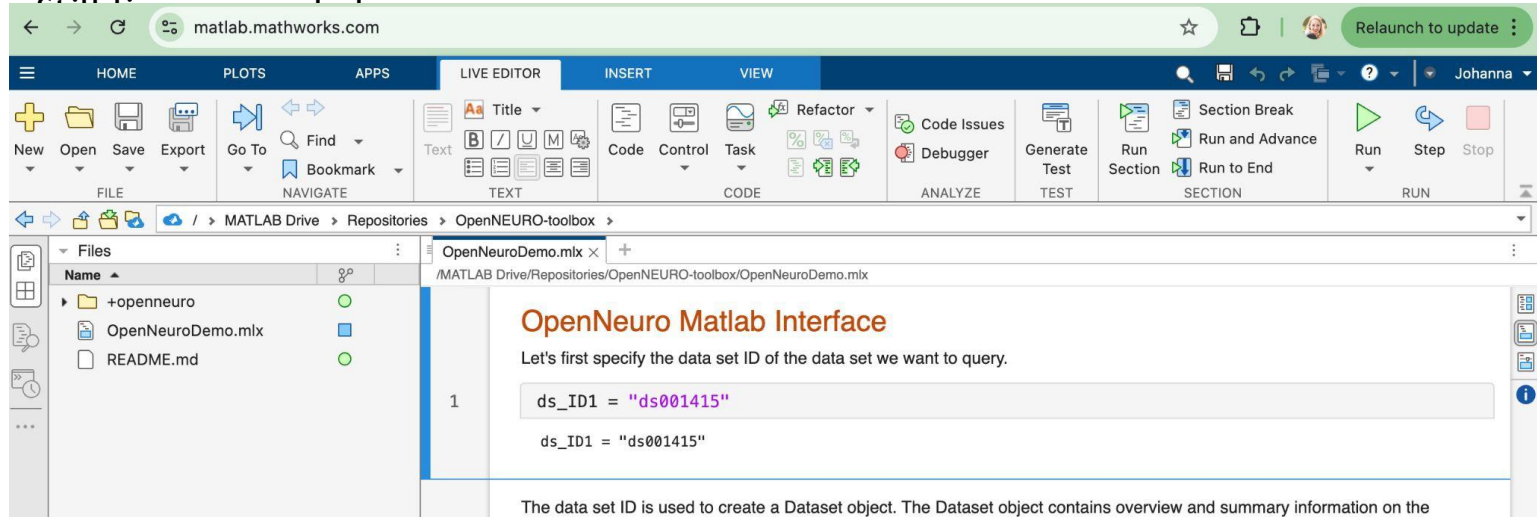
```
% startup.m
addpath(genpath('src'));
addpath('scripts');
disp('Project environment initialized.');
```

Set favourite commands



Matlab Live Scripts

- Cell based matlab scripts (.mlx)
- Can be opened in the browser (Matlab online)
 - Allows to share and execute code



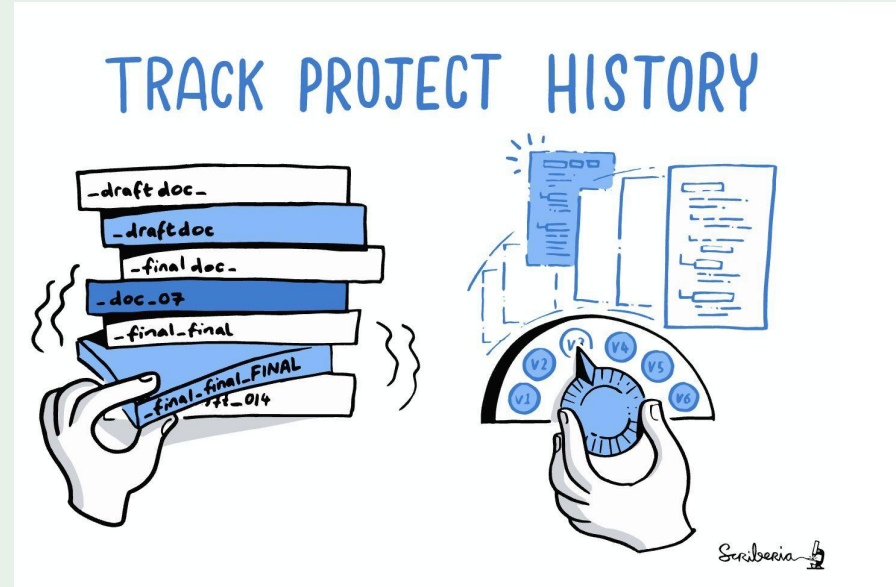
Set up how to compile/run your code

- Run via VS code
- Run via the command line/terminal
- Run via Jupyter notebook
- Run via a (other) IDE
- Run on a cluster/cloud ...

Setting up a coding project on a cluster

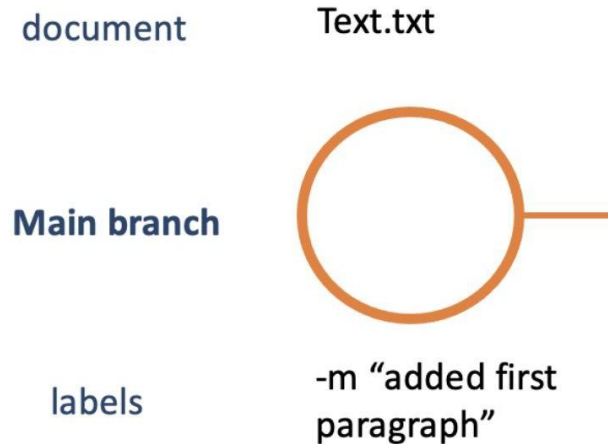
- Rsync of folders between cluster and local (images, tables etc)
- Ssh into cluster via vscode
- Sbatch

Committing



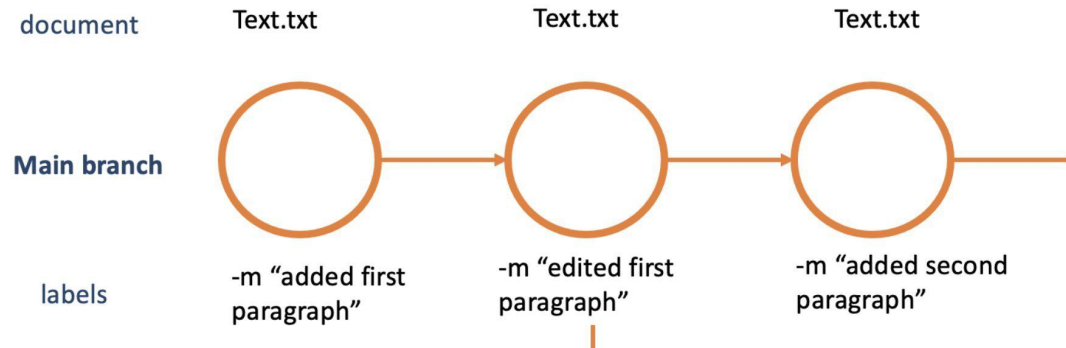
What is a commit?

- **Commits:** annotated snapshot of changes in a document



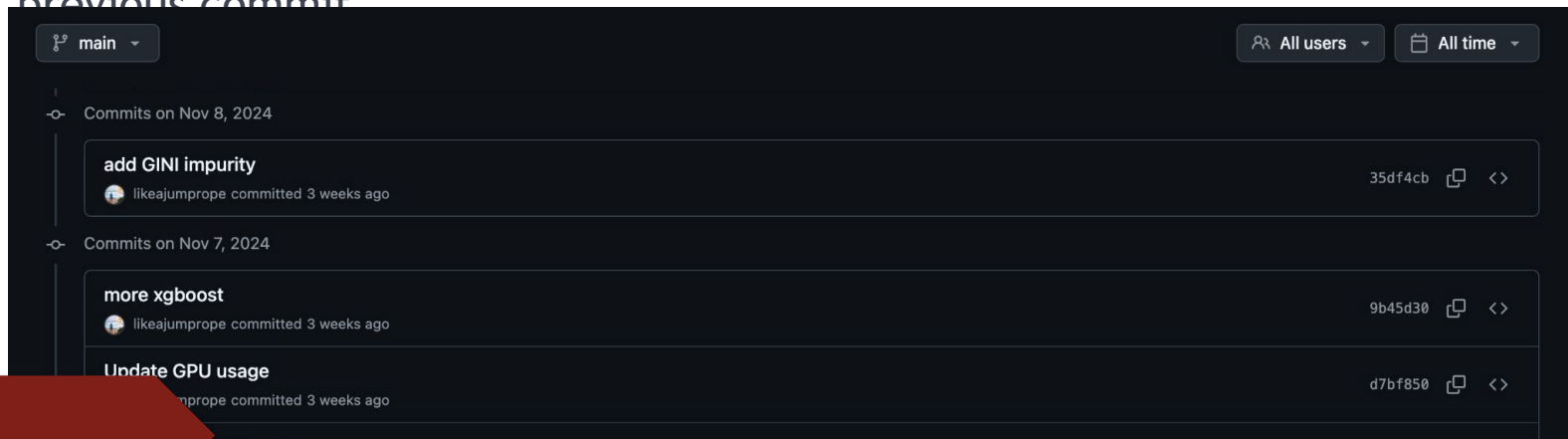
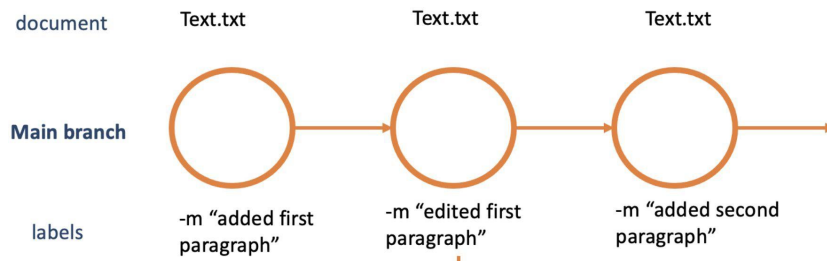
What is a commit?

- **Commits:** annotated snapshot of changes in a document

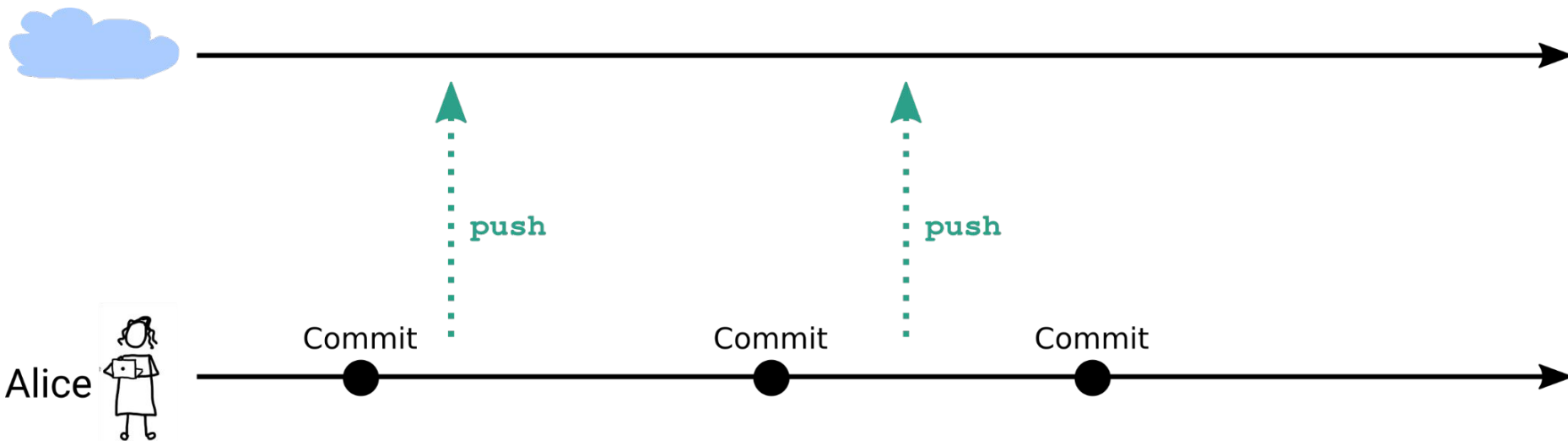


Git and Github: tracking your files, scripts and code

- Version control for files and scripts
- Ability to revert back to a previous commit

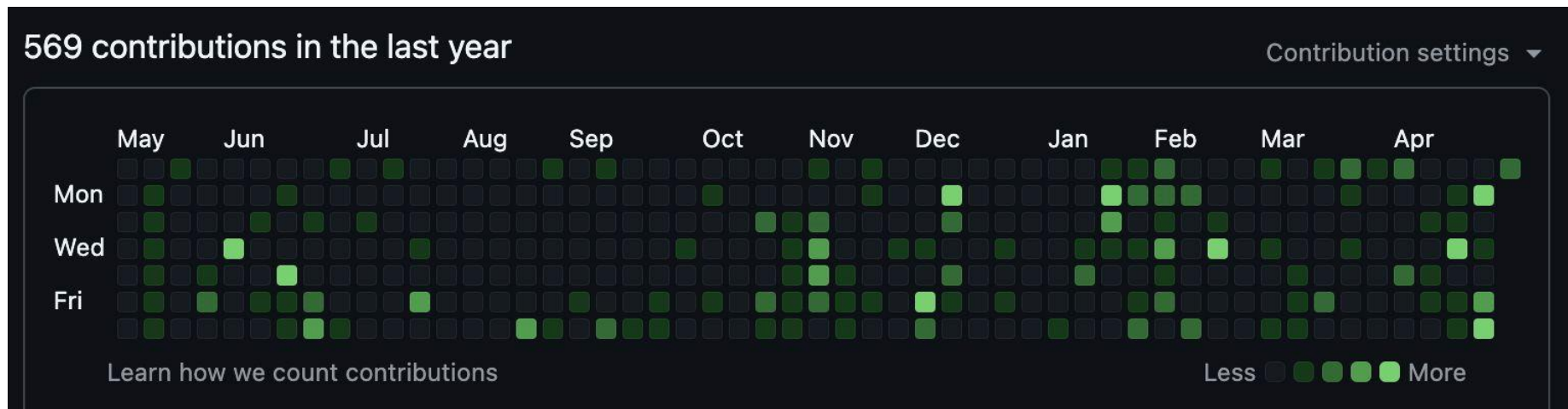


Git workflow for a single developer



Git and Github: tracking your files, scripts and code

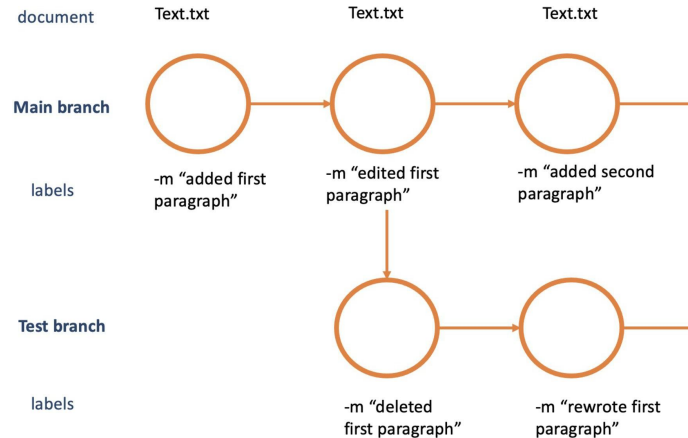
- Commit history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

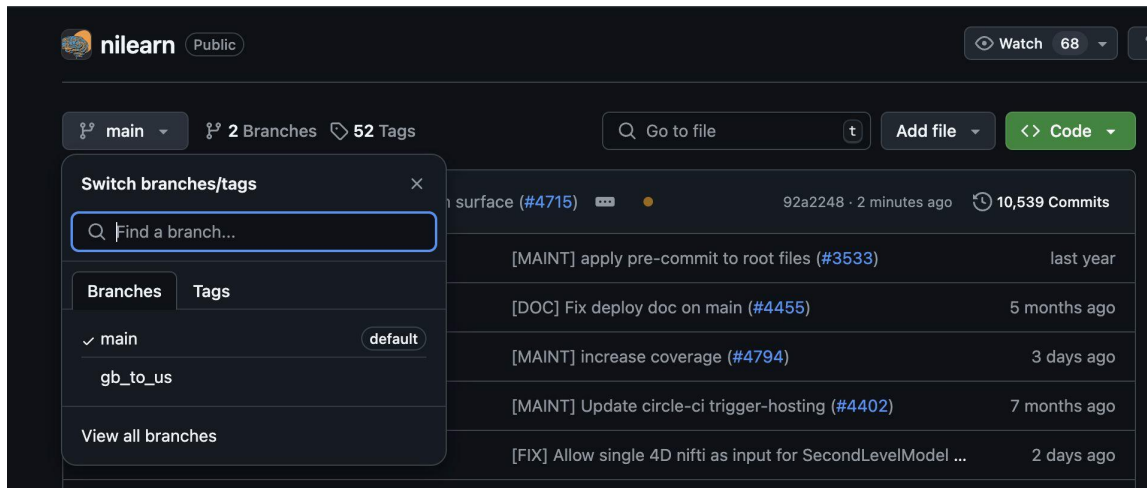
- **Branch:** Diversion point of repository and history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

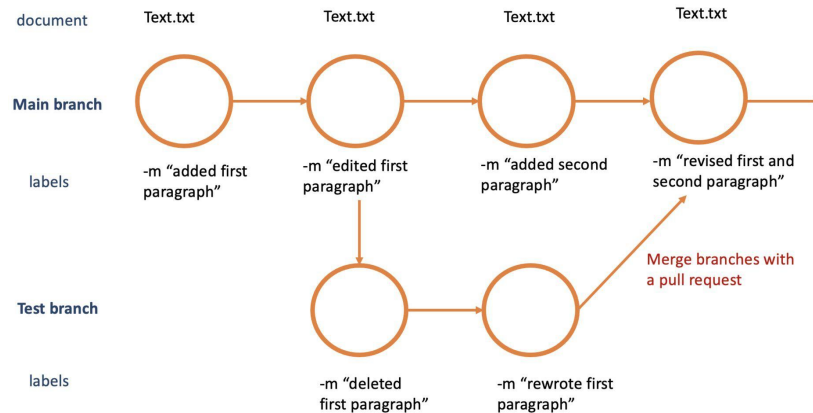
- **Branch:** Diversion point of repository and history



Git and Github: tracking your files, scripts and code

Collaboration (github online)

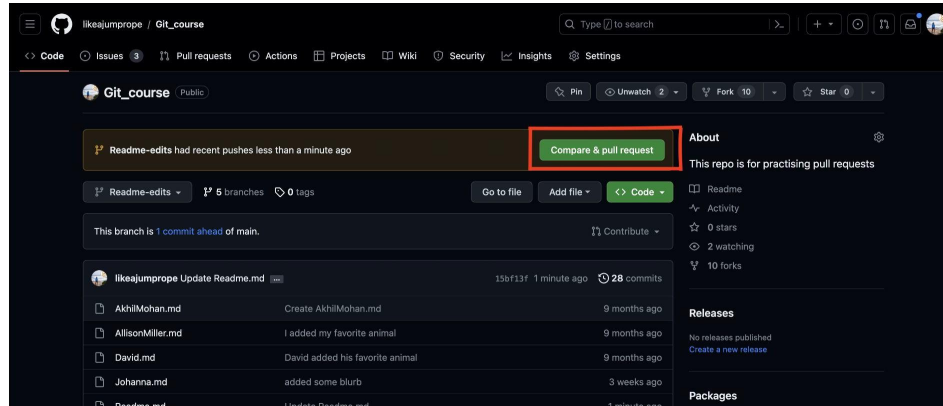
- **merging:** Unifying divergent branches, integrating changes
- Merging between branches is performed using a **pull request**.



Git and Github: tracking your files, scripts and code

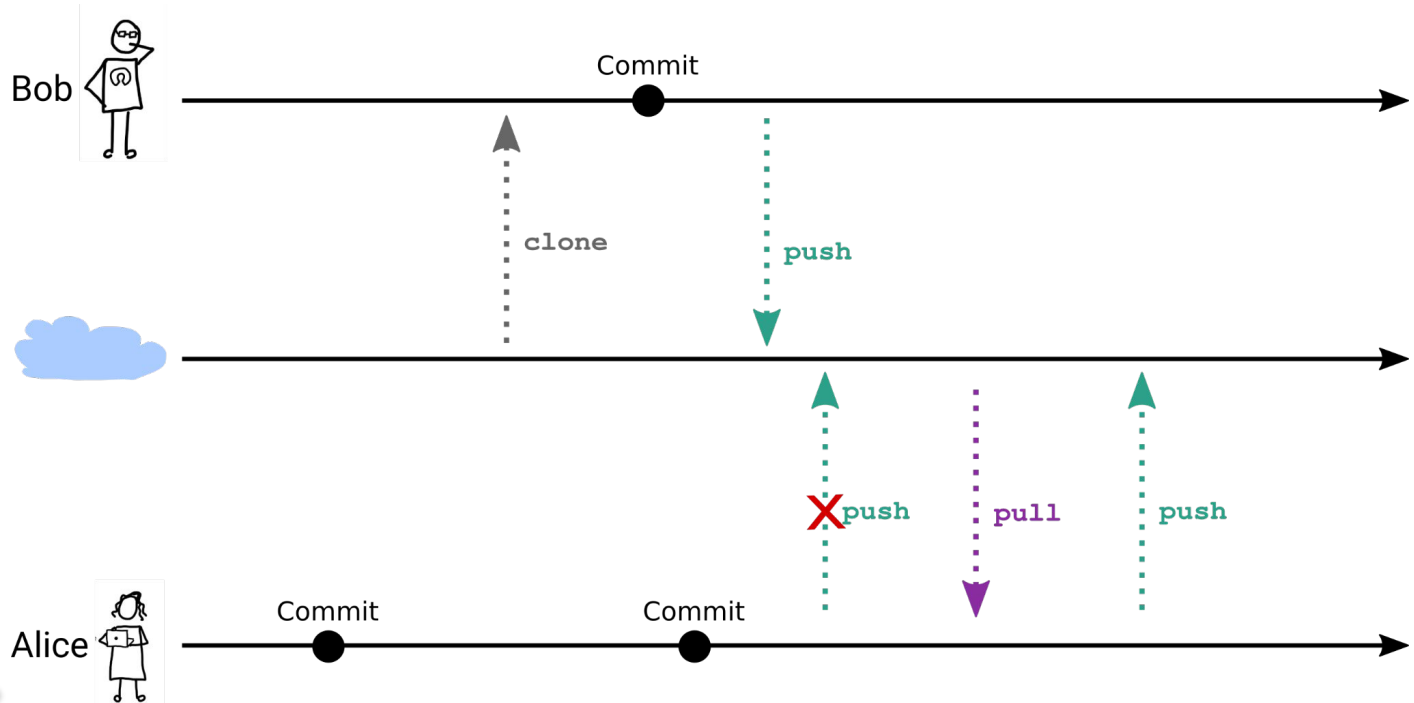
Collaboration (github online)

- **merging:** Unifying divergent branches, integrating changes
- Merging between branches is performed using a **pull request**.



Git and Github: tracking your files, scripts and code

Collaboration (github online)



Git and Github: tracking your files, scripts and code

Collaboration (github online)

- **Fork:** make a copy of someone else's repository under your account.
- Repositories on Github are divided into two categories
 - Yours (you have writing permission)
 - Other people's (you don't)
- **Forking** creates a copy of someone else's repository that you own (aka you have writing permission)

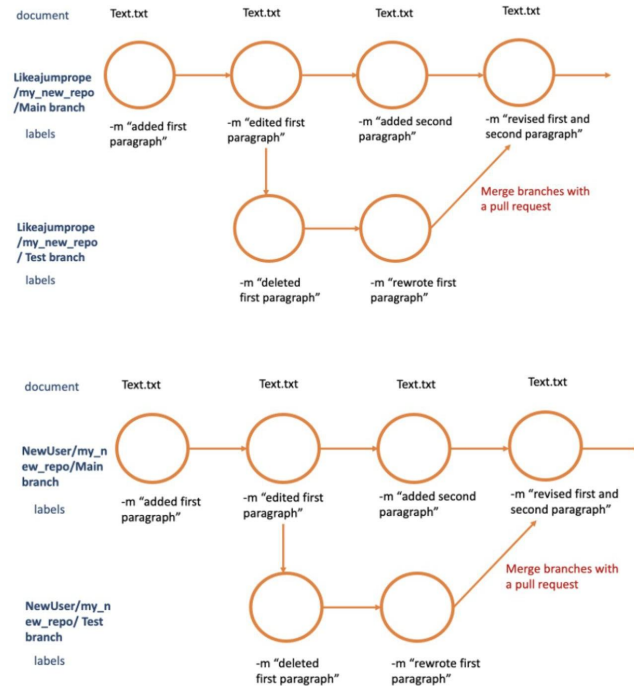


Git and Github: tracking your files, scripts and code

Collaboration (github online)

-

Forking



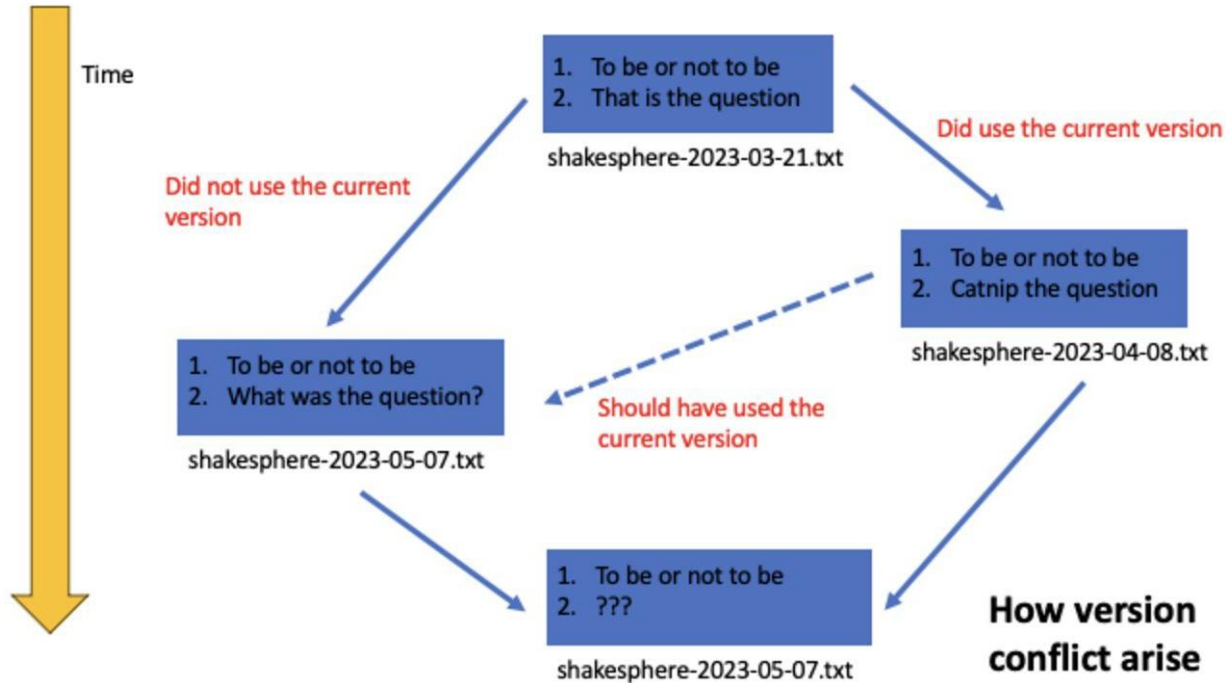
Why is it called pull request?

- Remember the separation between your and other people's repositories.
- In order to get your changes into someone else's repository, you have to request for them to be pulled in. This action is not up to you.
- You request a pull.
- The other person accepts, and pulls.

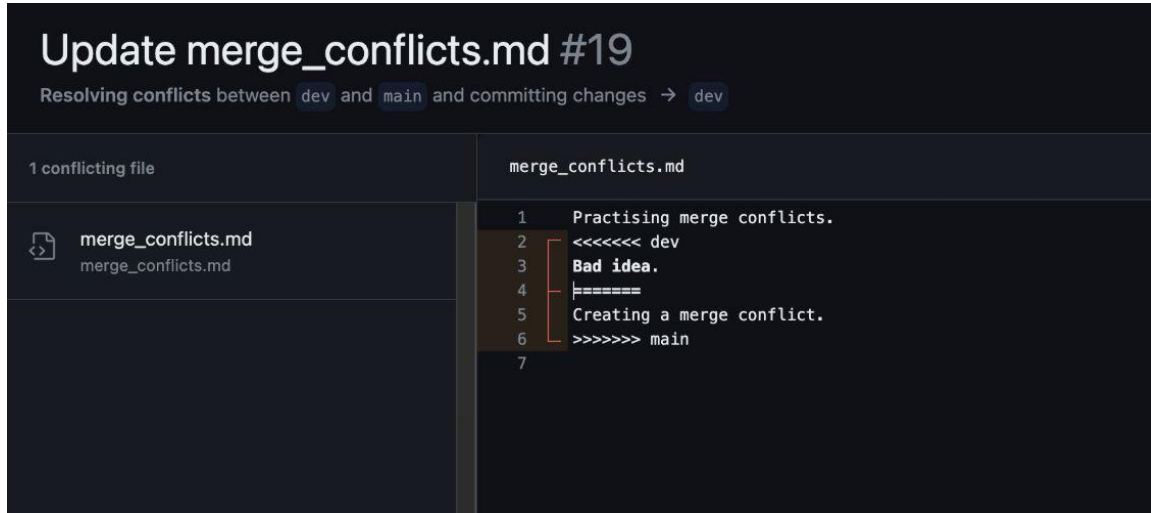


Fig. 191 Making your first pull request on GitHub. The Turing Way project illustration by Scriberia. Used under a CC-BY 4.0 licence. DOI: 10.5281/zenodo.3332807.

Merge conflicts



Resolving a merge conflict



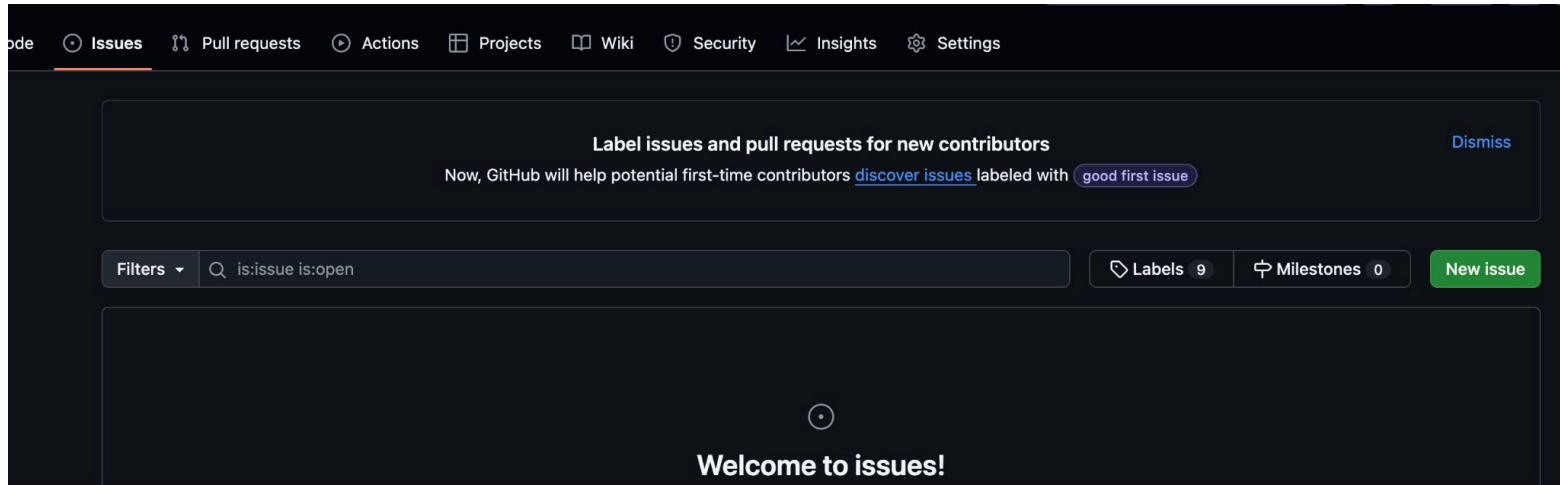
The screenshot shows a code editor interface with a dark theme. At the top, a title bar reads "Update merge_conflicts.md #19". Below it, a subtitle indicates the operation: "Resolving conflicts between dev and main and committing changes → dev". The main workspace is divided into two panes. The left pane, titled "1 conflicting file", shows a file named "merge_conflicts.md" with a file icon. The right pane shows the content of "merge_conflicts.md" with line numbers 1 through 7. The text content is as follows:

```
1 Practising merge conflicts.  
2 <<<<<< dev  
3 Bad idea.  
4 =====  
5 Creating a merge conflict.  
6 >>>>>> main  
7
```

A red bracket on the left side of the editor highlights the conflict region, spanning from line 2 to line 6. The lines are: "2 <<<<<< dev", "3 Bad idea.", "4 =====", "5 Creating a merge conflict.", and "6 >>>>>> main".

Issues on Github

Issues on GitHub are a built-in tool for tracking tasks, enhancements, bugs, and other project-related activities. They provide a way for developers, collaborators, and even external users to communicate and manage the work associated with a GitHub repository.



Issues and collaboration on Github online

Issues are the way of communicating on Github

- Issues follow markdown format
- PRs can be referenced in Issues (and vice versa)
- Usually one PR responds to an issue

